IACR Communications in Cryptology
ISSN 3006-5496, Vol. 3, No. 1, 40 pages.

https://doi.org/10.62056/ahsg893y6

zkExp: Zero-Knowledge Succinct Exponentiation Proofs

Biniyam Deressa and M. Anwar Hasan

University of Waterloo, Waterloo, Canada

Abstract. We present zkExp (Zero-Knowledge Succinct Exponentiation Proofs), the first zero-knowledge proof system achieving asymptotically efficient bounds for batched exponentiation: $\tilde{O}(k\ell)$ prover time, $O(1)$ verification time, and constant-size (160-256B) proofs. For statements $y_i = g^{x_i}$ $(i = 1, \ldots, k)$ with private exponents x_i, zkExp introduces four innovations to overcome long-standing scalability barriers: (1) trace-based square-and-multiply encoding, (2) lazy sumcheck for exponentiation constraints, (3) hybrid FFT decomposition reducing memory from $O(\ell)$ to $O(\sqrt{\ell})$, and (4) sliding-window batching enabling single-proof aggregation via KZG commitments. The protocol is computationally sound under the (q, ℓ)-Generalized Diffie-Hellman Exponent (GDHE) assumption and achieves computational zero-knowledge in the random oracle model. Proofs remain 160-256 bytes regardless of parameter sizes, with constant verification (3.5ms). For 4096-bit exponents, prover overhead is 16.3× (dropping to 1.35× in 1000-batch settings), while Ethereum verification costs ∼267k gas for 1000 exponentiations, 10× cheaper than ECDSA, with memory consumption below 1.1MB. zkExp is the first protocol to match theoretical lower bounds for exponentiation proofs while enabling practical deployment in zero-knowledge rollups, anonymous credentials, and on-chain threshold cryptography.

Keywords: Zero-Knowledge Proofs · Succinct Non-Interactive Arguments · Exponentiation · Polynomial Commitments · Batch Verification

1 Introduction

Modern cryptographic systems increasingly rely on proving knowledge of secret exponents in discrete logarithm relations. Applications ranging from RSA signature verification [RSA78] and anonymous credentials [CL01] to blockchain oracles [ZCC+16] and threshold cryptographic services require demonstrating that $y_i = g^{x_i}$ for secret exponents $\boldsymbol{x} = (x_1, \ldots, x_k) \in \mathbb{Z}_q^k$ and known group element $g \in \mathbb{G}$, where $\mathbb{G}$ is a cyclic group of prime order q. The underlying single-instance relation $\mathcal{R}_{\text{exp}} = \{((g, y), x) : y = g^x,\ x \in [0, 2^\ell)\}$ and its batch counterpart

$$\mathcal{R}_{\text{batch}} = \left\{\left((g, (y_i)_{i=1}^k), (x_i)_{i=1}^k\right) : y_i = g^{x_i},\ x_i \in [0, 2^\ell)\ \forall i \in [k]\right\}$$

underpin the security of numerous cryptographic protocols, with soundness grounded in the discrete logarithm assumption.

The classical approach to proving such relations employs independent Schnorr proofs [Sch91], yielding linear costs in both proof size and verification time with respect to the number of statements k. While adequate for small-scale deployments, this linear scaling becomes prohibitive in high-throughput environments. Consider VDF-based blockchain systems like Chia [CP19], where thousands of exponentiation proofs are generated regularly, imposing verification overhead exceeding 100k group operations per

E-mail: bderessa@uwaterloo.ca (Biniyam Deressa), ahasan@uwaterloo.ca (M. Anwar Hasan)

Received: 2025-10-06 Accepted: 2025-12-02

epoch for light clients and creating substantial storage requirements on the network. Similarly, large-scale anonymous credential systems and threshold cryptographic services demand efficient batching mechanisms to maintain practical performance [CHP07]. The pervasive nature of these scaling challenges spans virtually every domain of modern cryptographic infrastructure, from blockchain systems and enterprise PKI to IoT networks and financial trading platforms.

The apparent solution lies in general-purpose zero-knowledge succinct non-interactive arguments of knowledge (zk-SNARKs) [Gro16, GWC19], which offer constant-size proofs and verification times. However, when applied to exponentiation, these systems encounter inherent circuit overhead that creates structural inefficiencies. Representing an ℓ-bit exponentiation as an arithmetic circuit necessitates $\Omega(\ell)$ multiplication gates, resulting in $\Omega(k\ell)$ constraints for k exponentiations. The prover must then perform FFT operations over polynomials of degree $O(k\ell)$, yielding $O(k\ell \log(k\ell))$ computational complexity and memory requirements that rapidly become infeasible for cryptographic parameters such as $\ell = 2048$ [WTS+18].

Recent advances in batch verification have made significant progress toward addressing these challenges. The work of Hoffmann, Hubáček, and Ivanova [HHI25] introduced sophisticated batching protocols that achieve constant-size proofs and verification times for classical proof-of-exponentiation schemes. Their bucket protocol, in particular, demonstrates remarkable practical efficiency by distributing statements across 2^k buckets and applying optimized aggregation techniques, achieving order-of-magnitude improvements for large batch sizes. However, these approaches operate within honest-verifier settings without zero-knowledge guarantees and fundamentally optimize existing proof-of-exponentiation schemes rather than developing new proof systems. Their verification complexity remains tied to the structure of the underlying protocols.

Our Contributions. We introduce *zkExp*, the first zero-knowledge proof system to simultaneously achieve all three asymptotic bounds for batched exponentiation: $\widetilde{O}(k\ell)$ prover time, $O(1)$ verifier time, and $O(1)$ proof size. zkExp is designed from first principles to exploit the algebraic structure of repeated squaring, rather than optimizing over existing proof-of-exponentiation protocols or general-purpose SNARKs.

Our construction integrates four technical innovations that overcome the longstanding barriers to efficient zero-knowledge proofs for exponentiation. First, we develop a trace-based encoding of exponentiation that captures the full computation as a polynomial over a multiplicative subgroup, avoiding the constraint overhead of arithmetic circuits. Second, we design novel *lazy sumcheck* protocols that verify squaring and multiplication relations without incurring polynomial degree blowup, ensuring constant-degree constraints even across long squaring chains. Third, we introduce a hybrid FFT decomposition strategy that partitions exponentiation traces into $\sqrt{\ell}$ blocks, significantly reducing prover memory while preserving arithmetic efficiency. Finally, we propose a sliding-window batching protocol that aggregates arbitrarily many exponentiations into a single proof, enabling $O(1)$ verifier complexity via a unified KZG batch opening.

zkExp achieves computational zero-knowledge in the random oracle model via the Fiat-Shamir transformation (Theorem 5), while the interactive variant achieves perfect honest-verifier zero-knowledge (HVZK). Soundness is proven under the (q, ℓ)-Generalized Diffie-Hellman Exponent (GDHE) assumption via a tight reduction to the binding property of KZG commitments (Theorem 3). Our implementation demonstrates practical scalability: generating a proof for 1,000 exponentiations of 2048-bit exponents takes under 20ms on commodity hardware, with verification requiring less than 80k Ethereum gas.

To our knowledge, zkExp is the first protocol to match the theoretical lower bounds for batch exponentiation proofs established by Abe et al. [AFK87] while achieving full zero-knowledge, constant-size proofs, and constant-time verification under standard cryp-

tographic assumptions.

Technical Overview. zkExp exploits the algebraic structure of exponentiation via repeated squaring through polynomial commitments. For an exponent x with binary representation $(b_0, b_1, \ldots, b_{\ell-1})$, the computation g^x proceeds through intermediate values $v_0, v_1, \ldots, v_\ell$ where $v_0 = 1$, $v_\ell = g^x$, and $v_{j+1} = v_j^2 \cdot g^{b_j}$ for each $0 \leq j < \ell$. Rather than constraining this computation through arithmetic circuits, we encode the trace as polynomial evaluations over the multiplicative subgroup $\mathcal{D}_\ell$, enabling efficient verification through a single KZG polynomial commitment opening. Our key innovation, the *lazy sumcheck protocol*, resolves polynomial degree explosion by verifying squaring and multiplication constraints through bounded-degree quotient polynomials. This maintains constant verification complexity independent of exponent length while naturally extending to batched exponentiations via random linear combinations.

Organization. Section 2 positions zkExp within batch verification and zero-knowledge proof literature, while Section 3 introduces the cryptographic foundations, notation, and algebraic tools underlying our construction. Section 4 presents the core protocol with trace-based encoding, polynomial constraints, and lazy sumcheck verification, which Section 5 then extends to batched exponentiations via hybrid FFT decomposition and sliding-window aggregation. Section 6 formalizes the computational models, setup assumptions, and cryptographic hardness assumptions, while Section 7 establishes completeness, soundness, knowledge soundness, and zero-knowledge through formal reductions. Section 8 reports implementation and performance results, including concrete parameter evaluation and on-chain gas analysis where k denotes the batch size of exponentiations. Section 9 concludes with applications to blockchain and privacy-preserving protocols, while Section 6.5 provides extended cryptanalysis.

2 Related Work

We position zkExp within three distinct research trajectories: classical batch verification techniques, general-purpose zero-knowledge systems, and specialized algebraic arguments, each contributing essential insights to the problem space.

2.1 Classical Batch Verification Techniques

The systematic verification of batched exponentiation proofs traces its origins to Schnorr's seminal protocol for discrete logarithm relations [Sch91]. Bellare and Neven's foundational work [BNN09] established aggregation methods for independent Schnorr proofs through random linear combinations, achieving $O(k)$ verification complexity at the cost of $\Theta(k)$ communication. Subsequent advances by Boneh et al. [BDN18] leveraged BLS signature aggregation to reduce communication to $O(\log k)$ through pairing operations, though verification complexity remained intrinsically linear in batch size.

Recent breakthroughs demonstrate remarkable efficiency gains within specialized domains. Wesolowski's protocol [Wes19] achieves constant-size proofs for restricted exponentiations ($y = g^{2^t}$) in groups of unknown order, while Pietrzak-inspired constructions [BHR+21, Rot21] employ statistical soundness amplification for improved verification. The current state-of-the-art is epitomized by Hoffmann, Hubáček, and Ivanova [HHI25], whose sophisticated bucket methodology enables constant-size proofs and $O(1)$ verification time for 10^5+ exponentiations through hierarchical aggregation structures.

Despite these advances, classical approaches exhibit three fundamental limitations: (1) prover complexity remains strictly linear in batch size, (2) zero-knowledge guarantees are

architecturally unattainable, and (3) they cannot exploit modern polynomial commitment frameworks for asymptotic improvements.

Remark 1 (Limits of Schnorr-based batching). Batching Schnorr proofs via random linear combinations [BNN09] proves knowledge of the *aggregate* witness $x^* = \sum_{i=1}^{k} \alpha_i x_i$ for verifier-chosen scalars $\{\alpha_i\}$, establishing only $\prod_{i=1}^{k} y_i^{\alpha_i} = g^{x^*}$. It does *not* prove the per-instance relations $y_i = g^{x_i}$: no extractor can recover individual witnesses $(x_1, \ldots, x_k)$, and range guarantees $x_i \in [0, 2^\ell)$ cannot be enforced without additional mechanisms that reintroduce linear proof size. zkExp instead proves the full per-instance batch relation $\mathcal{R}_{\text{batch}} = \{((g_i, y_i)_{i=1}^k, (x_i)_{i=1}^k) : y_i = g_i^{x_i},\ x_i \in [0, 2^\ell)\}$ with per-instance witness extraction (Theorem 4) and constant proof size.

2.2 General-Purpose Zero-Knowledge Systems

The landscape of general-purpose proof systems bifurcates into transparent constructions avoiding trusted setups and succinct pairing-based systems requiring structured reference strings.

Transparent paradigms: Bulletproofs [BBB+18] express exponentiation via constraint systems, requiring $O(k\ell)$ constraints for k exponentiations of ℓ-bit scalars, yielding $O(\log(k\ell))$ proof size. STARKs [BSBHR18] provide post-quantum security but require $\Omega(\ell)$ field operations per exponentiation due to FFT-based constraints. DARK commitments [BFS20] enable $O(1)$ proof sizes but rely on class group assumptions with computationally expensive operations. Spartan [Set20] achieves $O(\log(k\ell))$ proofs with $O(k\ell \log(k\ell))$ prover complexity.

Succinct SNARKs: Systems including Groth16 [Gro16], Plonk [GWC19], and Marlin [CHM+20] achieve the theoretical optimum of $O(1)$ proof size and verification time. However, when compiling exponentiation to arithmetic circuits, they necessitate $\Omega(\ell)$ multiplication gates per operation due to the bit-decomposition requirements of square-and-multiply algorithms. This results in $\Omega(k\ell)$ constraints for k exponentiations, yielding $O(k\ell \log(k\ell))$ prover complexity with memory requirements that become prohibitive at security levels like $\ell = 2048$. Recursive proof composition techniques like Hyrax [WTS+18] and Halo/Nova [BGH19, KST22] amortize verification costs but fail to address the inherent circuit representation overhead.

2.3 Specialized Algebraic Arguments

Specialized protocols exploit algebraic structure to circumvent general-purpose limitations. Hoffmann et al.'s LMPAZK and QESAZK protocols [HKR19] achieve $O(\log n)$ communication complexity for linear and quadratic relations but provide only honest-verifier zero-knowledge (HVZK) and lack native batching mechanisms.

Zero-Knowledge Limitations in Specialized Protocols: Protocols achieving high efficiency through direct algebraic structure exploitation face inherent barriers to full zero-knowledge. Hoffmann et al.'s LMPAZK and QESAZK protocols [HKR19] achieve $O(\log n)$ complexity by exposing compact linear and quadratic relations between public values and witness elements. In their original form, these provide only honest-verifier zero-knowledge, as the challenge-response patterns reveal structural information exploitable by malicious verifiers. Standard conversion techniques like Fiat-Shamir transformation require careful analysis to preserve soundness properties when algebraic relations become observable through programmed random oracles.

Similarly, Wesolowski's construction [Wes19] operates in groups of unknown order using sequential squaring computations that inherently expose timing and intermediate states. While highly efficient for verifiable delay functions, this sequential structure was not designed to provide zero-knowledge guarantees, as simulation would require

breaking the fundamental sequentiality assumptions underlying the protocol's security. Both approaches optimize verification efficiency by utilizing computational structures that full zero-knowledge protocols must carefully conceal through polynomial masking and simulation techniques.

Batch Proof Techniques: Recent advances in SNARK batching include PLONK's quotient polynomials [GWC19] and Aurora's domain partitioning [BCR+19] for constraint parallelization, while Boneh et al. [BBF19] pioneered linear combination batching. SnarkPack [GMN22] aggregates proofs via random linear combinations but lacks cross-proof consistency. Our work introduces a novel synthesis: *hybrid domain decomposition* for exponentiation-specific FFT optimization and *sliding-window batching* with difference polynomials, achieving the first constant-size batched proofs for $y = g^x$ with $\widetilde{\mathcal{O}}(k\ell)$ prover complexity.

Trace-based proof systems constitute a complementary approach. The zkMaP protocol [DH25] demonstrates efficient matrix verification via polynomial commitments, yet suffers from polynomial degree blowup when modelling iterative squaring operations. Recent multi-scalar multiplication SNARKs (MSM-SNARKs) achieve $\widetilde{O}(k\ell)$ prover complexity through endomorphism optimizations [FKSX25] but maintain superlinear dependence on exponent bit-length.

2.4 Positioning of zkExp

zkExp represents a paradigm shift by simultaneously transcending the limitations of these approaches. Where classical batch verification optimizes existing protocols, zkExp leverages polynomial commitments to achieve asymptotic improvements. Where general-purpose SNARKs incur circuit compilation overhead, zkExp employs direct algebraic representation of repeated squaring chains. Crucially, our protocol resolves the polynomial degree explosion that obstructs trace-based methods through novel lazy sumcheck techniques.

Under the q-Strong Diffie-Hellman assumption, zkExp achieves $\widetilde{O}(k\ell)$ prover time alongside $O(1)$ verifier time and constant proof size, establishing the first protocol to simultaneously attain these three complexity bounds. zkExp provides computational zero-knowledge in the random oracle model via the Fiat-Shamir transformation, surpassing the HVZK guarantees of specialized approaches; its interactive variant achieves perfect honest-verifier zero-knowledge (HVZK). With $\widetilde{O}(k\ell)$ prover complexity matching the theoretical lower bound [Gol04], to our knowledge, zkExp is the first protocol to achieve $\widetilde{O}(k\ell)$ prover time, $O(1)$ verification time, and $O(1)$ proof size with computational zero-knowledge for batched exponentiation.

Table 1: Asymptotic comparison of batched exponentiation proof systems for k exponentiations with ℓ-bit secret exponents. $\widetilde{O}$ notation hides polylogarithmic factors. ZK Type and Model columns distinguish between honest-verifier ZK (HVZK), full ZK against malicious verifiers, and the interaction/setup model, respectively.

Scheme	Proof Size	Prover Time	Verifier Time	Setup	Assumption	ZK Type	Model
Schnorr [Sch91] (batched [BNN09])	$O(k)$	$O(k)$	$O(k)$	None	DL	HVZK*	Σ/FS
Random Subsets [HHI25, BHR+21]	$O(\lambda)$	$O(\lambda m)$	$O(\lambda)$	None	DL	None†	Challenge-Resp.
Random Exponents [HHI25, Rot21]	$O(1)$	$O(\lambda m)$	$O(1)$	None	Low-order	None†	Challenge-Resp.
Bucket Protocol [HHI25]	$O(1)$	$O(m + 2^k\lambda)$	$O(1)$	None	Low-order	None†	Challenge-Resp.
Hybrid Protocol [HHI25]	$O(1)$	$O(\lambda(m + 3\lambda))$	$O(1)$	None	Low-order	None†	Challenge-Resp.
Bulletproofs [BBB+18]	$O(\log(k\ell))$	$O(k\ell)$	$O(k + \log(k\ell))$	None	DL	Full	ROM-FS
Groth16 [Gro16]	$O(1)$	$O(k\ell \log(k\ell))$	$O(1)$	Trusted	q-PKE	Full	NIZK‡
Plonk [GWC19]	$O(1)$	$O(k\ell \log(k\ell))$	$O(1)$	Universal	q-SDH	Full	ROM-FS
LMPAZK [HKR19]	$O(\log n)$	$O(n)$	$O(\log n)$	None	DL	HVZK*	Interactive
QESAZK [HKR19]	$O(\log n)$	$O(n)$	$O(\log n)$	None	DL	HVZK*	Interactive
Wesolowski [Wes19]	$O(1)$	$O(t/\log t)$	$O(1)$	None	Unknown order	None	Standard
zkExp (this work)	$O(1)$	$\widetilde{O}(k\ell)$	$O(1)$	**Trusted**	(q,ℓ)-**GDHE**	**Full**§	**ROM-FS**

Legend: ZK Type — *HVZK* = Honest-Verifier Zero-Knowledge (secure only against non-malicious verifiers); *Full* = computational zero-knowledge against malicious verifiers; *None* = no zero-knowledge property. **Model** — Σ*/FS* = Sigma protocol, non-interactive via Fiat-Shamir; *ROM-FS* = Random

Oracle Model with Fiat-Shamir; *NIZK* = Non-Interactive Zero-Knowledge (can be instantiated in ROM or CRS model); *Interactive* = requires interaction; *Standard* = standard model (no random oracle or setup); *Challenge-Resp.* = challenge-response proof without ZK.

* Systems marked with HVZK* can achieve full ZK via Fiat-Shamir in the ROM but are originally HVZK in interactive form.

† These are proof-of-exponentiation schemes for *public* exponentiations, not zero-knowledge proofs.

‡ Groth16 achieves NIZK in the Common Reference String (CRS) model; practical instantiation uses Fiat-Shamir in ROM.

§ The non-interactive (ROM-FS) zkExp variant achieves full computational ZK against malicious verifiers; its underlying interactive variant achieves only perfect HVZK.

3 Preliminaries

We establish the cryptographic foundations for our zero-knowledge exponentiation protocol. All constructions operate in pairing-friendly elliptic curve groups $\mathbb{G}_1$, $\mathbb{G}_2$, $\mathbb{G}_T$ of prime order $p = \Theta(2^\lambda)$ with security parameter λ, equipped with a non-degenerate bilinear map $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$. Public generators $g_1 \in \mathbb{G}_1$ and $g_2 \in \mathbb{G}_2$ are fixed, and we assume the discrete logarithm problem is hard in both source groups.

3.1 Discrete Exponentiation Relations

The core cryptographic relation $\mathcal{R}_{\text{exp}} = \{((g, y), x) : y = g^x,\ x \in [0, 2^\ell)\}$ underpins numerous fundamental cryptographic primitives, including digital signatures, anonymous credentials, and threshold cryptosystems. As modern cryptographic applications scale to handle massive transaction volumes, they increasingly require efficient batch verification of thousands of such relations while preserving zero-knowledge properties—a challenging requirement that our protocol directly addresses through novel algebraic techniques.

3.2 Cryptographic Assumptions

Our protocol's security relies on two standard assumptions: the discrete logarithm problem in bilinear groups, and a structured variant of the q-Strong Diffie-Hellman assumption adapted to polynomial constraints over multiplicative subgroups. Complete definitions and security analysis appear in Section 6.

3.3 Polynomial Commitment Schemes

Our construction builds upon the Kate-Zaverucha-Goldberg (KZG) polynomial commitment scheme [KZG10], which provides the essential capability to commit to polynomials and later prove their evaluations at specific points. The setup algorithm $\mathsf{Setup}(1^\lambda, d)$ generates a structured reference string $\mathsf{srs} = (g_1^{s^i}, g_2^{s^i})_{i=0}^d$ for a secret value $s \xleftarrow{\$} \mathbb{F}_p$, enabling commitments to polynomials of degree up to d.

To commit to a polynomial $f \in \mathbb{F}_p[X]$, the prover computes $C_f = g_1^{f(s)}$, leveraging the structure of the reference string. When opening the commitment at a point z, the prover provides both the evaluation $y = f(z)$ and a succinct proof $\pi = g_1^{q(s)}$, where $q(X) = (f(X) - y)/(X - z)$ represents the quotient polynomial that exists precisely when the evaluation is computed honestly. The verifier confirms the opening by checking the pairing equation $e(\pi, g_2^s/g_2^z) = e(C_f/g_1^y, g_2)$, which algebraically enforces the quotient relationship in the exponent. The KZG scheme provides computational binding under the q-SDH assumption while delivering constant-size evaluation proofs regardless of polynomial degree.

3.4 Algebraic Tools for Efficient Verification

A fundamental component enabling our efficient batch verification approach is the vanishing polynomial $v_{\mathcal{D}_\ell}(X) = X^\ell - 1$ associated with the multiplicative subgroup $\mathcal{D}_\ell \subset \mathbb{F}_p^*$ of order ℓ, where $\ell \mid p-1$. This algebraic structure allows us to replace ℓ individual constraint checks with a single, elegantly batched verification step.

To confirm that a constraint polynomial $h(X)$ vanishes on all points of $\mathcal{D}_\ell$, we leverage the fundamental divisibility property: if h indeed vanishes on $\mathcal{D}_\ell$, then $h(X) = Q(X) \cdot v_{\mathcal{D}_\ell}(X)$ for some quotient polynomial $Q(X)$. Rather than checking $h(\omega^i) = 0$ for each $i \in \{0, \ldots, \ell-1\}$ individually, we verify the single quotient relation $h(\alpha) = Q(\alpha) \cdot v_{\mathcal{D}_\ell}(\alpha)$ at a random challenge point $\alpha \xleftarrow{\$} \mathbb{F}_p \setminus \mathcal{D}_\ell$. By the Schwartz-Zippel lemma, this approach achieves soundness error bounded by $\deg(h)/(p-\ell)$, which is negligible for appropriately chosen parameters.

The practical implementation of these polynomial operations leverages fast Fourier transforms, achieving $O(\ell \log \ell)$ computational complexity for operations over structured domains. Our hybrid decomposition strategy further enhances efficiency by organizing evaluations into $\sqrt{\ell}$-sized blocks, reducing peak memory requirements from $O(\ell)$ to $O(\sqrt{\ell})$ while preserving the favorable time complexity. This optimization proves crucial for deployment in resource-constrained environments without sacrificing the protocol's performance characteristics.

3.5 Trace-Based Computation

Following the established paradigm of modern succinct arguments [BCG+13, CHM+20], our approach encodes computational traces as polynomial evaluations over the structured domain $\mathcal{D}_\ell$. For an exponentiation computation involving ℓ steps, we construct a trace polynomial $V(X)$ that satisfies $V(\omega^i) = v_i$, where v_i represents the intermediate value computed at step i of the square-and-multiply algorithm.

The correctness of this trace encoding is ensured through a carefully designed system of polynomial constraints that capture the essential properties of modular exponentiation. The squaring consistency constraint requires $W(\omega^i) = V(\omega^i)^2$, ensuring that squared intermediate values are computed correctly at each step. The binary constraint enforces $B(\omega^i) \in \{0, 1\}$ through the polynomial relation $B(\omega^i)(B(\omega^i) - 1) = 0$, guaranteeing that exponent bits are properly represented. Finally, the recurrence constraint mandates $V(\omega^{i+1}) = W(\omega^i) \cdot g^{B(\omega^i)}$, which elegantly captures the controlled multiplication step that distinguishes between squaring-only and square-and-multiply operations based on the current exponent bit.

Our zero-knowledge argument system built upon this trace-based foundation achieves perfect completeness, ensuring that honest provers can always generate valid proofs for true statements. Computational soundness follows from our (q, ℓ)-GDHE assumption, while computational zero-knowledge is guaranteed through careful polynomial masking techniques that statistically hide the underlying witness information. The Fiat-Shamir transformation converts our interactive protocol into a non-interactive variant suitable for practical deployment, maintaining all security properties while enabling efficient verification in distributed settings.

4 zkExp: Zero-Knowledge Exponentiation Protocol

We present zkExp, a zero-knowledge argument system for discrete exponentiation that transforms the verification of $y = g^x$ into polynomial constraint satisfaction over structured domains. Our construction achieves constant-size proofs with logarithmic prover complexity

through algebraic batching of trace-based encodings and lazy constraint verification, advancing the state-of-the-art in efficient exponentiation verification.

4.1 Algebraic Foundation and Trace Construction

The discrete exponentiation relation forms the mathematical foundation of our protocol. Given a cyclic group $\mathbb{G}$ of prime order p with generator g, we define the relation

$$\mathcal{R}_{\text{exp}} = \{((g, y), x) : y = g^x,\ x \in [0, 2^\ell)\}$$

Our objective is to construct a zero-knowledge argument system that enables a prover possessing secret exponent x to convince a verifier that $y = g^x$ without revealing any information about x beyond its validity.

For exponent bit-length ℓ, we work with the multiplicative subgroup $\mathcal{D}_\ell = \{\omega^0, \omega^1, \ldots, \omega^{\ell-1}\} \subset \mathbb{F}_p^*$, where ω is a primitive ℓ-th root of unity satisfying $\omega^\ell = 1$ and $\omega^i \neq 1$ for $1 \leq i < \ell$. We require $\ell \mid (p-1)$ to ensure such a root exists. The associated vanishing polynomial $Z_\ell(X) = X^\ell - 1$ plays a central role in constraint verification by encoding the property that polynomials vanish on the entire evaluation domain.

The Lagrange interpolation framework provides the theoretical foundation for polynomial reconstruction over $\mathcal{D}_\ell$. For each $k \in \{0, 1, \ldots, \ell-1\}$, the k-th Lagrange basis polynomial is defined as

$$L_k(X) = \prod_{j \neq k} \frac{X - \omega^j}{\omega^k - \omega^j} = \frac{Z_\ell(X)}{(X - \omega^k) \cdot Z'_\ell(\omega^k)},$$

where $Z'_\ell(X) = \ell X^{\ell-1}$ denotes the formal derivative and $Z'_\ell(\omega^k) = \ell\omega^{k(\ell-1)}$. Note that ℓ is invertible in $\mathbb{F}_p$ since $\ell < p$ by construction. These polynomials satisfy the fundamental interpolation property $L_k(\omega^j) = \delta_{k,j}$, enabling unique polynomial reconstruction from point evaluations.

Exponentiation Trace Construction. For an exponent $x \in \mathbb{Z}_p$ with $x < 2^\ell$, we consider its unique binary representation $x = \sum_{j=0}^{\ell-1} b_j \cdot 2^j$ where each $b_j \in \{0, 1\}$. An exponentiation trace for input (g, x) is defined as a sequence $\mathcal{T} = (v_0, v_1, \ldots, v_\ell) \in \mathbb{G}^{\ell+1}$ satisfying initialization $v_0 = 1_\mathbb{G}$, recurrence $v_{j+1} = v_j^2 \cdot g^{b_j}$ for all $j \in \{0, 1, \ldots, \ell-1\}$, and termination $v_\ell = g^x$. The mathematical correctness follows from the trace invariant $v_j = g^{x \bmod 2^j}$ for all $j \in \{0, 1, \ldots, \ell\}$. This invariant can be established by strong induction on j. The base case $v_0 = 1_\mathbb{G} = g^0$ holds by initialization. For the inductive step, assuming the invariant holds for some j, we have

$$v_{j+1} = v_j^2 \cdot g^{b_j} = (g^{x \bmod 2^j})^2 \cdot g^{b_j} = g^{2(x \bmod 2^j) + b_j} = g^{x \bmod 2^{j+1}},$$

where the final equality follows from the binary representation structure.

4.2 Polynomial Encoding and Constraint System

We transform the discrete trace values into polynomial representations over $\mathcal{D}_\ell$. Given exponentiation trace $\mathcal{T} = (v_0, v_1, \ldots, v_\ell)$ with corresponding bit sequence $(b_0, b_1, \ldots, b_{\ell-1})$, we construct four fundamental trace polynomials: the *value polynomial* $V(X) \in \mathbb{F}_p[X]$ with $V(\omega^j) = v_j$ for $j \in \{0, 1, \ldots, \ell-1\}$, the *squared-value polynomial* $W(X) \in \mathbb{F}_p[X]$ with $W(\omega^j) = v_j^2$, the *bit polynomial* $B(X) \in \mathbb{F}_p[X]$ with $B(\omega^j) = b_j$, and the *next-value polynomial* $R(X) \in \mathbb{F}_p[X]$ with $R(\omega^j) = v_{j+1}$ for $j \in \{0, \ldots, \ell-2\}$ and $R(\omega^{\ell-1}) = y$. These polynomial representations are unique with degree at most $\ell - 1$ due to interpolation over ℓ distinct points. The prover generates KZG commitments: $\mathsf{Com}(V), \mathsf{Com}(W), \mathsf{Com}(B)$, $\mathsf{Com}(R) \in \mathbb{G}_1$ over a structured reference string supporting degree 2ℓ. To ensure computational zero-knowledge in the random oracle model (via Fiat-Shamir), we employ

polynomial masking by sampling random field elements $\delta_V, \delta_W, \delta_B, \delta_R \xleftarrow{\$} \mathbb{F}_p$ and defining masked polynomials $V'(X) = V(X) + \delta_V \cdot Z_\ell(X)$, with analogous definitions for $W'(X), B'(X), R'(X)$.

4.3 Constraint System Design

Fix a prime p and an integer ℓ with $\ell \mid (p-1)$. Let $\omega \in \mathbb{F}_p^\times$ be a primitive ℓ-th root of unity and set $\mathcal{D}_\ell = \{\omega^t : t = 0, \ldots, \ell-1\} \subset \mathbb{F}_p^\times$, with vanishing polynomial $Z_\ell(X) = \prod_{t=0}^{\ell-1}(X - \omega^t)$. Let $H \subset \mathbb{F}_p^\times$ be a multiplicative subgroup containing the protocol base g with $|H| \geq 2^\ell$, ensuring the map $x \mapsto g^x$ is injective on $[0, 2^\ell)$ without modular wrap-around. Take polynomials $V, W, B, R \in \mathbb{F}_p[X]$ of degree strictly less than ℓ; their values on $\mathcal{D}_\ell$ uniquely determine them by interpolation. The constraint system below enforces that these polynomials encode a correct square-and-multiply trace for some ℓ-bit exponent.

The correctness of exponentiation traces is verified through six polynomial constraints. The squaring consistency constraint $h_1(X) \coloneqq W(X) - V(X)^2$ ensures squared values are computed correctly. The binary consistency constraint $h_2(X) \coloneqq B(X) \cdot (B(X) - 1)$ guarantees $B(\omega^j) \in \{0, 1\}$ for all evaluation points. The recurrence consistency constraint $h_3(X) \coloneqq R(X) - W(X) \cdot \big(1 + (g-1) \cdot B(X)\big)$ preserves the multiplicative structure. The trace-linkage constraint $h_4(X) \coloneqq \sum_{j=0}^{\ell-2} L_j(X) \cdot \big(V(\omega^{j+1}) - W(\omega^j) \cdot g^{b_j}\big)$ enforces step-wise multiplicative continuity, where $b_j = B(\omega^j) \in \{0, 1\}$ and $L_j(X)$ denotes the j-th Lagrange basis polynomial over $\mathcal{D}_\ell$ satisfying $L_j(\omega^k) = \delta_{j,k}$ for $k \in \{0, \ldots, \ell-1\}$. This ensures h_4 vanishes on $\{\omega^0, \ldots, \omega^{\ell-2}\}$ if and only if $V(\omega^{j+1}) = W(\omega^j) \cdot g^{b_j}$ holds for each $j \in [0, \ell-2]$, without imposing a spurious wrap-around constraint at $j = \ell - 1$. Since $\deg(L_j) = \ell - 1$ and the coefficient polynomials have degree $< \ell$, we have $\deg(h_4) \leq 2\ell - 1$, consistent with the SRS degree bound.

The initialization constraint $h_{\text{init}}(X) \coloneqq V(\omega^0) - 1$ establishes proper starting conditions, while the output binding constraint $h_{\text{out}}(X) \coloneqq R(\omega^{\ell-1}) - y$ verifies the final result. Together, these six constraints ensure that any satisfying polynomial tuple (V, W, B, R) encodes a unique valid square-and-multiply execution terminating at the claimed output y. The formal completeness and soundness of this constraint system is established in Section 7.3.

Lazy Verification. When a constraint polynomial $h_i(X)$ vanishes on $\mathcal{D}_\ell$, it is divisible by $Z_\ell(X)$, yielding quotient polynomial $Q_i(X) = h_i(X)/Z_\ell(X)$. The verifier samples random $\alpha \xleftarrow{\$} \mathbb{F}_p \setminus \mathcal{D}_\ell$ and checks the quotient relations $h_i(\alpha) = Q_i(\alpha) \cdot Z_\ell(\alpha)$ for $i \in \{1, 2, 3\}$. The trace-linkage constraint h_4 together with the boundary conditions $h_{\text{init}}, h_{\text{out}}$ are enforced in the full batched protocol of Section 5, while Algorithm 3 checks the primary quotient relations only. By the Schwartz–Zippel lemma [Sch80], if any $h_i(X)$ does not vanish identically on $\mathcal{D}_\ell$, the equality fails with probability at least $1 - \deg(h_i)/|\mathbb{F}_p|$, which is negligible for cryptographic field sizes.

4.4 Protocol Specification

The complete zkExp protocol integrates the trace encoding, polynomial commitment, and constraint verification components into a coherent zero-knowledge argument system operating over bilinear groups $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ of prime order p. We present the protocol using three formal algorithms. Algorithms 1–3 constitute the *single-instance building block*; the asymptotic efficiency advantages materialize in the batch protocol of Section 5, which extends this construction to k exponentiations with $O(1)$ proof size and verifier cost.

Setup Phase: The trusted setup procedure (Algorithm 1) generates the structured reference string and evaluation domain parameters required for polynomial commitments and constraint verification. The setup supports exponents up to bit-length ℓ by constructing

a KZG SRS of degree 2ℓ, ensuring compatibility with our constraint system's degree requirements.

Proof Generation: The prover algorithm (Algorithm 2) begins by computing the square-and-multiply trace and extracting the binary representation of the secret exponent. It then constructs the four trace polynomials via Lagrange interpolation, applies zero-knowledge masking, and commits to the masked polynomials using KZG. After generating the Fiat-Shamir challenge, the prover constructs three primary constraint polynomials h_1, h_2, h_3 for explicit verification. The trace-linkage constraint h_4 together with the boundary conditions $h_{\text{init}}, h_{\text{out}}$ are incorporated into the overall constraint system governing the trace. Their complete enforcement appears in the batched construction of Section 5, while Algorithm 2 prepares the corresponding trace commitments and quotient relations used in the verification procedure. The prover then computes quotients with respect to the vanishing polynomial and generates opening proofs for all polynomial evaluations at the challenge point.

Verification Process: The verifier algorithm (Algorithm 3) reconstructs the Fiat-Shamir challenge and verifies the integrity of all polynomial openings through batch KZG verification. The core verification logic checks that the three primary quotient relations hold at the challenge point. The trace-linkage constraint h_4 together with the boundary conditions $h_{\text{init}}, h_{\text{out}}$ are incorporated through the global trace relation enforced in the batched protocol of Section 5; the full soundness argument (Theorems 3 and 4) is therefore established for the complete zkExp system.

Algorithm 1 $\mathsf{zkExp.Setup}(1^\lambda, \ell)$

Require: Security parameter λ, exponent bit-length ℓ
Ensure: Public parameters pp

1: Generate bilinear group:
   $\mathcal{G} = (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, p, e)$
   where p is prime with $\lceil \log_2 p \rceil \geq 2\lambda$
2: Construct KZG SRS: $\mathsf{srs}_1 = \{g_1^{\tau^i}\}_{i=0}^{2\ell-1}$, $\mathsf{srs}_2 = \{g_2, g_2^\tau\}$ ▷ Degree $\geq 2\ell - 1$
3: Compute primitive ℓ-th root of unity $\omega \in \mathbb{F}_p^*$ ▷ $\ell \mid (p-1)$
4: Define evaluation domain $\mathcal{D}_\ell = \{\omega^j\}_{j=0}^{\ell-1}$
5: Set vanishing polynomial $Z_\ell(X) = X^\ell - 1$
6: **return** $\mathsf{pp} = (\mathcal{G}, \mathsf{srs}, \mathcal{D}_\ell, Z_\ell)$ *where* $\mathsf{srs} = (\mathsf{srs}_1, \mathsf{srs}_2)$

Batch KZG Operations. We use two batch KZG primitives throughout the algorithms.

- $\mathsf{KZG.BatchOpen}(\mathsf{srs}, \alpha, \{f_i\}_{i=1}^n)$: Given SRS, evaluation point $\alpha \in \mathbb{F}_p$, and polynomials $f_1, \ldots, f_n$, derives aggregation scalars $r_1, \ldots, r_n \xleftarrow{\mathcal{H}} \mathbb{F}_p$ via Fiat-Shamir from all commitments $\{g_1^{f_i(\tau)}\}$, and returns a single opening proof $\pi_{\text{open}} = g_1^{h(\tau)} \in \mathbb{G}_1$ where $h(X) = \big(\sum_i r_i f_i(X) - \sum_i r_i f_i(\alpha)\big)/(X - \alpha)$.
- $\mathsf{KZG.BatchVerify}(\mathsf{srs}, \{(C_i, \alpha, v_i)\}_{i=1}^n, \pi_{\text{open}})$: Recomputes the same aggregation scalars $\{r_i\}$ and checks $e\big(C_{\text{agg}} - g_1^{v_{\text{agg}}}, g_2\big) = e\big(\pi_{\text{open}}, g_2^{\tau-\alpha}\big)$ where $C_{\text{agg}} = \prod_i C_i^{r_i}$ and $v_{\text{agg}} = \sum_i r_i v_i$.

Both primitives reduce n independent KZG openings to a single pairing check, preserving soundness under the (q, ℓ)-GDHE assumption (Theorem 3).

Remark 2 (Quotient commitments after challenge). The quotient polynomials Q_1, Q_2, Q_3 are committed *after* the Fiat-Shamir challenge α is derived from the trace commitments C_V, C_W, C_B, C_R. This does not harm soundness: each $Q_j = h_j / Z_\ell$ is *uniquely determined* by the already-committed trace polynomials V', W', B', R' and the public constraint

Algorithm 2 $\mathsf{zkExp.Prove}(\mathsf{pp}, g, y, x)$

Require: Public parameters pp, generator $g \in \mathbb{G}$, result $y \in \mathbb{G}$, exponent $x \in \mathbb{Z}_p$
Ensure: Zero-knowledge proof π

Parse $\mathsf{pp} = (\mathcal{G}, \mathsf{srs}, \mathcal{D}_\ell, Z_\ell)$
Fix group embedding: $g_{\text{field}} \leftarrow \iota(g) \in \mathbb{F}_p$ ▷ Injective map $\iota : \mathbb{G} \to \mathbb{F}_p$
Compute binary expansion $(b_0, \ldots, b_{\ell-1}) \leftarrow \mathsf{Bin}(x, \ell)$
Initialize trace: $v_0 \leftarrow 1_{\mathbb{G}}$
for $j \leftarrow 0$ to $\ell - 1$ **do**
    $v_{j+1} \leftarrow v_j^2 \cdot g^{b_j}$ ▷ Square-and-multiply
end for
assert $v_\ell = y$ ▷ Validate trace correctness
▷ Construct trace polynomials via Lagrange interpolation over $\mathcal{D}_\ell$

$$V(\omega^j) = v_j, \qquad W(\omega^j) = v_j^2, \qquad B(\omega^j) = b_j, \qquad R(\omega^j) = \begin{cases} v_{j+1}, & j < \ell - 1 \\ y, & j = \ell - 1 \end{cases}$$

Sample masks: $\delta_V, \delta_W, \delta_B, \delta_R \xleftarrow{\$} \mathbb{F}_p$ ▷ For computational zero-knowledge
Construct masked polynomials and commit:

$$\begin{aligned} V'(X) &\leftarrow V(X) + \delta_V \cdot Z_\ell(X), & C_V &\leftarrow \mathsf{KZG.Commit}(\mathsf{srs}_1, V') \\ W'(X) &\leftarrow W(X) + \delta_W \cdot Z_\ell(X), & C_W &\leftarrow \mathsf{KZG.Commit}(\mathsf{srs}_1, W') \\ B'(X) &\leftarrow B(X) + \delta_B \cdot Z_\ell(X), & C_B &\leftarrow \mathsf{KZG.Commit}(\mathsf{srs}_1, B') \\ R'(X) &\leftarrow R(X) + \delta_R \cdot Z_\ell(X), & C_R &\leftarrow \mathsf{KZG.Commit}(\mathsf{srs}_1, R') \end{aligned}$$

Compute Fiat-Shamir challenge (see Section 4.5):
$\alpha \leftarrow \mathcal{H}(\texttt{0x01}, g, y, C_V, C_W, C_B, C_R) \bmod p$
while $\alpha \in \mathcal{D}_\ell$ **do**
    $\alpha \leftarrow \mathcal{H}(\texttt{0x02}, \alpha) \bmod p$
end while
Construct constraints and quotients:

$$\begin{aligned} h_1(X) &\leftarrow W'(X) - V'(X)^2, & Q_1(X) &\leftarrow h_1(X)/Z_\ell(X) \\ h_2(X) &\leftarrow B'(X) \cdot (B'(X) - 1), & Q_2(X) &\leftarrow h_2(X)/Z_\ell(X) \\ h_3(X) &\leftarrow R'(X) - W'(X) \cdot (1 + B'(X) \cdot (g_{\text{field}} - 1)), & Q_3(X) &\leftarrow h_3(X)/Z_\ell(X) \end{aligned}$$

Commit quotients:

$$C_{Q_1} \leftarrow \mathsf{KZG.Commit}(\mathsf{srs}_1, Q_1), \quad C_{Q_2} \leftarrow \mathsf{KZG.Commit}(\mathsf{srs}_1, Q_2), \quad C_{Q_3} \leftarrow \mathsf{KZG.Commit}(\mathsf{srs}_1, Q_3)$$

Evaluate polynomials at α:
$\boldsymbol{eval} \leftarrow \{V'(\alpha), W'(\alpha), B'(\alpha), R'(\alpha), Q_1(\alpha), Q_2(\alpha), Q_3(\alpha)\}$
Generate batch opening proof:
$\boldsymbol{\pi}_{\text{open}} \leftarrow \mathsf{KZG.BatchOpen}(\mathsf{srs}, \alpha, \{V', W', B', R', Q_1, Q_2, Q_3\})$
return $\pi \leftarrow (C_V, C_W, C_B, C_R, C_{Q_1}, C_{Q_2}, C_{Q_3}, \boldsymbol{eval}, \boldsymbol{\pi}_{\text{open}})$

definitions. By the Schwartz–Zippel lemma, a cheating prover who deviates from the correct Q_j is detected with overwhelming probability at the evaluation point α, since $h_j(\alpha) \neq Q_j(\alpha) \cdot Z_\ell(\alpha)$ would require α to be a root of a non-zero polynomial of degree at most 2ℓ, which occurs with probability at most $2\ell/|\mathbb{F}_p| = \mathsf{negl}(\lambda)$.

The protocol achieves perfect completeness since honest executions always satisfy all

Algorithm 3 zkExp.Verify(pp, g, y, π)

Require: Public parameters pp, generator $g \in \mathbb{G}$, result $y \in \mathbb{G}$, proof π
Ensure: accept or reject

Parse $\mathsf{pp} = (\mathcal{G}, \mathsf{srs}, \mathcal{D}_\ell, Z_\ell)$
Parse $\pi = (C_V, C_W, C_B, C_R, C_{Q_1}, C_{Q_2}, C_{Q_3}, \boldsymbol{eval}, \boldsymbol{\pi}_{\text{open}})$
Parse $\boldsymbol{eval} = (v', w', b', r', q_1, q_2, q_3)$
Compute group embedding: $g_{\text{field}} \leftarrow \iota(g) \in \mathbb{F}_p$ ▷ Same mapping as prover
Compute $\alpha \leftarrow \mathcal{H}(\texttt{0x01}, g, y, C_V, C_W, C_B, C_R) \bmod p$ ▷ y binds proof to this statement; KZG.BatchVerify is a generic subroutine—statement binding occurs here
while $\alpha \in \mathcal{D}_\ell$ **do**
    $\alpha \leftarrow \mathcal{H}(\texttt{0x02}, \alpha) \bmod p$
end while
Verify batch opening: ▷ $O(1)$ pairings via batch KZG
$\mathsf{checks} \leftarrow \mathsf{KZG.BatchVerify}(\mathsf{srs}, \{(C_V, \alpha, v'), (C_W, \alpha, w'), (C_B, \alpha, b'), (C_R, \alpha, r'), (C_{Q_1}, \alpha, q_1), (C_{Q_2}, \alpha, q_2), (C_{Q_3}, \alpha, q_3)\}, \boldsymbol{\pi}_{\text{open}})$
if $\mathsf{checks} = \mathsf{false}$ **then return** reject
end if
Compute $z_\alpha \leftarrow \alpha^\ell - 1$
Verify quotient relations:

$$\hat{h}_1 \leftarrow w' - (v')^2, \qquad \hat{h}_2 \leftarrow b' \cdot (b' - 1), \qquad \hat{h}_3 \leftarrow r' - w' \cdot \big(1 + b' \cdot (g_{\text{field}} - 1)\big)$$

if $\hat{h}_1 \neq q_1 \cdot z_\alpha$ **or** $\hat{h}_2 \neq q_2 \cdot z_\alpha$ **or** $\hat{h}_3 \neq q_3 \cdot z_\alpha$ **then return** reject
end if
return accept ▷ Checks h_1, h_2, h_3; trace-linkage h_4 enforced in Algorithm 5.

constraints by construction. The proof size is constant, consisting of seven group elements for polynomial commitments, seven field elements for evaluations, and the batch opening proof, yielding total size $|\pi| = O(1)$ independent of the exponent length.

4.5 Fiat-Shamir Transformation

The non-interactive variant employs the Fiat-Shamir transformation [FS86] with domain separation and collision avoidance. Challenge $\alpha \in \mathbb{F}_p$ is derived deterministically: first compute $\alpha \leftarrow \mathcal{H}(\texttt{0x01} \parallel g \parallel y \parallel C_V \parallel C_W \parallel C_B \parallel C_R) \bmod p$. If $\alpha \in \mathcal{D}_\ell$, iteratively rehash $\alpha \leftarrow \mathcal{H}(\texttt{0x02} \parallel \alpha) \bmod p$ until $\alpha \notin \mathcal{D}_\ell$ (expected iterations $\ell/p \ll 1$). Polynomial evaluations $\{V'(\alpha), W'(\alpha), B'(\alpha), R'(\alpha), Q_1(\alpha), Q_2(\alpha), Q_3(\alpha)\}$ follow from Lemma 2.

The random oracle $\mathcal{H} : \{0,1\}^* \to \mathbb{F}_p$ uses SHA-256 with modular reduction. Group elements employ compressed point representation (48 bytes for $\mathbb{G}_1$, 96 bytes for $\mathbb{G}_2$) and field elements use 32-byte big-endian encoding. Byte prefixes `0x01` and `0x02` ensure domain separation between primary challenge generation and collision resolution, preserving quotient polynomial verification soundness.

4.6 Proof Aggregation for Single Instance

Algorithm 2 produces seven commitments $\{C_V, C_W, C_B, C_R, C_{Q_1}, C_{Q_2}, C_{Q_3}\}$ with corresponding KZG openings $\{\pi_i\}_{i=1}^7$ at challenge α. This is an optional deployment optimization layered on top of Algorithm 2; the base protocol output remains the full proof object defined above. In deployment scenarios with persistent verification (e.g., blockchain smart contracts, verifiable databases), these commitments form part of the public statement

and are stored by the verifier. The prover transmits only the aggregated proof, achieving compact communication.

Batch Opening. The verifier derives aggregation coefficients $r_1, \ldots, r_7 \xleftarrow{\$} \mathbb{F}_p$ via Fiat-Shamir from the transcript, including all seven commitments to prevent bias. The prover computes:

$$C_{\text{agg}} = \prod_{i=1}^{7} C_i^{r_i} \in \mathbb{G}_1 \qquad \text{and} \qquad \pi_{\text{agg}} = \prod_{i=1}^{7} \pi_i^{r_i} \in \mathbb{G}_1$$

with aggregated evaluation $v_{\text{agg}} = \sum_{i=1}^{7} r_i \cdot v_i$ where $v_i = f_i(\alpha)$. Verification becomes:

$$e(\pi_{\text{agg}}, g_2^{\tau-\alpha}) = e(C_{\text{agg}}/g_1^{v_{\text{agg}}}, g_2)$$

where the verifier computes $g_2^{\tau-\alpha}$ from the trusted verification key, reducing from seven pairings to one.

Proof Size. The transmitted proof comprises $C_{\text{agg}} \in \mathbb{G}_1$ (48 bytes), $\pi_{\text{agg}} \in \mathbb{G}_1$ (48 bytes), and evaluation values $v_{\text{agg}}, \alpha \in \mathbb{F}_p$ (64 bytes), totaling 160 bytes. This assumes that commitments are pre-published as part of the statement. The verifier performs one multi-pairing check (equivalent to two standard pairings) plus field operations to compute $\{r_i\}$ and verify the aggregation. Zero-knowledge is preserved as polynomials remain masked by multiples of $Z_\ell(X)$. Soundness follows from Theorem 3. The batch protocol (Section 5) extends this construction to k exponentiations with constant 256-byte proofs via sliding-window aggregation.

5 Batch zkExp: Efficient Aggregation of Exponentiation Proofs

Building upon the single-instance zkExp protocol (Section 4), we present a batching technique that aggregates proofs for k distinct exponentiations $\{(g_i, y_i, x_i)\}_{i=1}^{k}$ while preserving the polynomial constraint framework. Our construction achieves $\widetilde{O}(k\ell)$ prover complexity and constant-size verification through hybrid domain decomposition for constraint parallelization and a sliding-window protocol for cross-instance consistency.

5.1 Hybrid Domain Decomposition

To reduce the $O(k\ell \log \ell)$ complexity of naive batching, we partition each exponentiation's evaluation domain $\mathcal{D}_\ell$ into $\sqrt{\ell}$ contiguous blocks $\mathcal{B}_m = \{\omega^{m\sqrt{\ell}}, \ldots, \omega^{(m+1)\sqrt{\ell}-1}\}$ for $0 \leq m < \sqrt{\ell}$. For each block $\mathcal{B}_m$, we compute local constraint polynomials $h_{i,j}^{(m)}(X)$ for $j \in \{1, 2, 3\}$ (recalling the three constraint types from Section 4.2) using size-$\sqrt{\ell}$ FFTs, construct the block vanishing polynomial $\mathcal{Z}_{\mathcal{B}_m}(X) = \prod_{\omega^j \in \mathcal{B}_m}(X - \omega^j)$, and derive local quotients $Q_{i,j}^{(m)}(X) = h_{i,j}^{(m)}(X)/\mathcal{Z}_{\mathcal{B}_m}(X)$.

The global quotient $Q_{i,j}(X)$ is assembled via composition of $\{Q_{i,j}^{(m)}\}_{m=0}^{\sqrt{\ell}-1}$, reducing the dominant FFT cost from $O(\ell \log \ell)$ to $O(\sqrt{\ell} \log \sqrt{\ell})$ per exponentiation while maintaining soundness properties.

5.2 Sliding-Window Batching Protocol

To enforce algebraic consistency across batched instances, we organize k exponentiations into overlapping windows $\mathcal{W}_t = \{\mathsf{Exp}_{(t-1)s+1}, \ldots, \mathsf{Exp}_{(t-1)s+w}\}$ for $1 \leq t \leq T$, where w

denotes the window width, $s = \lfloor w/2 \rfloor$ the stride, and $T = \lceil (k-w)/s \rceil + 1$ the number of windows (the final window may contain fewer than w exponentiations). Each non-boundary exponentiation appears in two consecutive windows, creating linkage points for consistency constraints.

Example 1 (Overlapping Window Structure)**.** Consider $k = 6$ exponentiations and a window configuration with $w = 3, s = 2$:

$$\begin{aligned}
&\texttt{Window 1}: && [\mathsf{Exp}_1,\ \mathsf{Exp}_2,\ \mathsf{Exp}_3] \\
&\texttt{Window 2}: && [\mathsf{Exp}_2,\ \mathsf{Exp}_3,\ \mathsf{Exp}_4] && \leftarrow \text{overlap: } \mathsf{Exp}_2,\ \mathsf{Exp}_3 \\
&\texttt{Window 3}: && [\mathsf{Exp}_3,\ \mathsf{Exp}_4,\ \mathsf{Exp}_5,\ \mathsf{Exp}_6] && \leftarrow \text{overlap: } \mathsf{Exp}_3,\ \mathsf{Exp}_4
\end{aligned}$$

The overlapping exponentiations $\mathsf{Exp}_2, \mathsf{Exp}_3$ (between windows $\mathcal{W}_1$ and $\mathcal{W}_2$, where $\mathcal{W}_t$ denotes the t-th window), and $\mathsf{Exp}_3, \mathsf{Exp}_4$ (between $\mathcal{W}_2$ and $\mathcal{W}_3$) induce consistency constraints across windows. Boundary elements such as Exp_1 and Exp_6 appear in a single window and do not compromise soundness.

Cross-Window Consistency. For shared exponentiations $\mathsf{Exp}_i \in \mathcal{W}_t \cap \mathcal{W}_{t+1}$, we enforce constraint equivalence via difference polynomials:

$$\delta_{i,j}^{(t)}(X) = h_{i,j}^{(t)}(X) - h_{i,j}^{(t+1)}(X) \quad \text{for } j \in \{1,2,3,4\}$$

where the four constraint types are: (1) squaring consistency, (2) binary consistency, (3) recurrence consistency, and (4) trace-linkage consistency (as defined in Lemma 2). The prover aggregates these differences and proves they vanish at the global challenge point α using KZG evaluation arguments, ensuring constraint polynomials remain consistent across overlapping windows.

The complete batch protocol formalizes the sliding-window aggregation through polynomial constraint batching. Algorithm 4 partitions the k exponentiations into T overlapping windows, enforces cross-window consistency via difference polynomials $\delta_{i,j}^{(t)}(X)$ for $j \in \{1,2,3,4\}$, and produces a single aggregated proof through KZG batch opening. Algorithm 5 performs an $O(k)$ statement-preprocessing phase followed by an $O(1)$ cryptographic-verification phase. The final proof achieves a constant 256-byte size through polynomial aggregation, independent of the batch size k.
For efficiency, Algorithm 5 uses the equivalent KZG verification form with precomputed helper element $C_{\text{verify}} = \text{srs}_2[1] \cdot g_2^{-\alpha_{\text{final}}} = g_2^{\tau - \alpha_{\text{final}}}$, i.e., $e(\pi_{\text{open}}, C_{\text{verify}}) = e(C_{Q_{\text{final}}} \cdot g_1^{-q_{\text{final}}}, g_2)$. This avoids a $\mathbb{G}_2$ scalar multiplication at the verifier; correctness follows directly from the KZG opening equation since C_{verify} encodes the standard denominator $g_2^{\tau - \alpha_{\text{final}}}$.

5.3 Proof Size and Complexity

The sliding-window protocol aggregates constraint polynomials across windows and overlapping instances via Fiat–Shamir hashing, producing a single committed quotient with constant-size opening. The transmitted batched proof is 256 bytes, independent of batch size k and window count T. Specifically, the proof object $(C_{Q_{\text{final}}}, \pi_{\text{open}}, C_{\text{verify}}, e_{\text{final}}, q_{\text{final}})$ comprises two $\mathbb{G}_1$ elements (96 bytes), one $\mathbb{G}_2$ element $C_{\text{verify}} = g_2^{\tau - \alpha_{\text{final}}}$ (96 bytes), and two $\mathbb{F}_p$ field elements (64 bytes), totalling exactly 256 bytes. The challenge α_{final} is recomputed by the verifier from the Fiat–Shamir transcript and is not transmitted. The prover supplies C_{verify} to avoid a $\mathbb{G}_2$ exponentiation during verification. Since the verifier recomputes α_{final} from the Fiat–Shamir transcript, any inconsistency between C_{verify} and the verifier-derived challenge would cause the KZG opening equation or the quotient relation at α_{final} to fail.

Algorithm 4 zkExp.BatchProve — Sliding-window batching with integrated constraints

Require: Public parameters pp, statements $\{(g_i, y_i)\}_{i=1}^{k}$, security bit-length ℓ, window width $w \geq 32$

Ensure: Batched proof π_{batch}

Parse $pp = (\mathcal{G}, \text{srs}, \mathcal{D}_\ell, Z_\ell, \omega)$; ω is a primitive ℓ-th root of unity

Set stride $s \leftarrow \lfloor w/2 \rfloor$, windows $T \leftarrow \lceil (k-w)/s \rceil + 1$

Statement binding

$H_{\text{stmt}} \leftarrow \mathcal{H}(\texttt{"zkExp-stmt"} \parallel \{(g_i, y_i)\}_{i=1}^{k}) \bmod p$ ▷ Binds all subsequent Fiat–Shamir challenges to this exact statement

Phase 1: Trace construction and interpolation

for $i = 1$ to k **do**

  Compute square-and-multiply trace with secret bits $(b_{i,0}, \ldots, b_{i,\ell-1})$: $v_{i,0} \leftarrow 1$, $v_{i,j+1} \leftarrow v_{i,j}^2 \cdot g_i^{b_{i,j}}$ for $j \in [0, \ell-1]$ ▷ $O(\ell)$ per exponentiation

  Interpolate over $\mathcal{D}_\ell$: $V_i(\omega^j) \leftarrow v_{i,j}$, $W_i(\omega^j) \leftarrow v_{i,j}^2$, $B_i(\omega^j) \leftarrow b_{i,j}$, $R_i(\omega^j) \leftarrow v_{i,j+1}$

  Fix group embedding: $g_{i,\text{field}} \leftarrow \iota(g_i) \in \mathbb{F}_p$ ▷ Injective map for constraint computation

end for

Phase 2: Window constraints via hybrid FFT (Sec. 5.1)

Derive global mixing scalars $(\rho_1, \rho_2, \rho_3, \rho_4, \rho_{\text{init}}, \rho_{\text{out}}) \leftarrow \mathcal{H}(\texttt{0x10} \parallel H_{\text{stmt}}) \bmod p$

for $t = 1$ to T **do**

  Define window $\mathcal{W}_t \leftarrow \{\mathsf{Exp}_{(t-1)s+1}, \ldots, \mathsf{Exp}_{\min((t-1)s+w,k)}\}$

  Derive instance weights $\{\gamma_{t,i}\}_{i \in \mathcal{W}_t} \leftarrow \mathcal{H}(\texttt{0x11} \parallel H_{\text{stmt}} \parallel t \parallel \{(g_i, y_i)\}_{i \in \mathcal{W}_t}) \bmod p$

  for $i \in \mathcal{W}_t$ **do**

    Using hybrid radix-2/radix-m FFT (Sec. 5.1), form constraint polynomials:

    $h_{i,1}^{(t)}(X) \leftarrow W_i(X) - V_i(X)^2$ (squaring)

    $h_{i,2}^{(t)}(X) \leftarrow B_i(X) \cdot (B_i(X) - 1)$ (binary)

    $h_{i,3}^{(t)}(X) \leftarrow R_i(X) - W_i(X) \cdot (1 + B_i(X) \cdot (g_{i,\text{field}} - 1))$ (recurrence)

    $h_{i,4}^{(t)}(X) \leftarrow \sum_{j=0}^{\ell-2} L_j(X) \, (V_i(\omega^{j+1}) - W_i(\omega^j) \cdot g_i^{b_{i,j}})$ (trace-linkage)

    if $t = 1$ **then**

    $h_{i,\text{init}}^{(t)}(X) \leftarrow V_i(\omega^0) - 1$ (initialization)

    end if

    if $t = T$ **then**

    $h_{i,\text{out}}^{(t)}(X) \leftarrow R_i(\omega^{\ell-1}) - y_i$ (output binding)

    end if

    Mask for ZK: for each applicable $j \in \{1, 2, 3, 4, \text{init}, \text{out}\}$, pick $\delta_{i,j}^{(t)} \xleftarrow{\$} \mathbb{F}_p$ and set $h_{i,j}^{(t)\prime}(X) \leftarrow h_{i,j}^{(t)}(X) + \delta_{i,j}^{(t)} \cdot Z_\ell(X)$

  end for

  Aggregate window polynomial:

  $T_t(X) \leftarrow \sum_{i \in \mathcal{W}_t} \gamma_{t,i} \cdot (\rho_1 h_{i,1}^{(t)\prime}(X) + \rho_2 h_{i,2}^{(t)\prime}(X) + \rho_3 h_{i,3}^{(t)\prime}(X) + \rho_4 h_{i,4}^{(t)\prime}(X)$

  $+ \mathbb{1}_{t=1} \rho_{\text{init}} h_{i,\text{init}}^{(t)\prime}(X) + \mathbb{1}_{t=T} \rho_{\text{out}} h_{i,\text{out}}^{(t)\prime}(X))$

  Commit $C_t \leftarrow \text{KZG.Commit}(\text{srs}, T_t)$

end for

Phase 3: Cross-window consistency (difference constraints)

Derive consistency mixing coefficients $\{\eta_{i,j}\}_{i,j} \leftarrow \mathcal{H}(\texttt{0x12} \parallel H_{\text{stmt}} \parallel \{C_t\}_{t=1}^{T}) \bmod p$

for $t = 1$ to $T - 1$ **do**

  For $i \in \mathcal{W}_t \cap \mathcal{W}_{t+1}$ and $j \in \{1, 2, 3, 4\}$, define $\Delta_{i,j}^{(t)}(X) \leftarrow h_{i,j}^{(t)\prime}(X) - h_{i,j}^{(t+1)\prime}(X)$

  Set $D_t(X) \leftarrow \sum_{i \in \mathcal{W}_t \cap \mathcal{W}_{t+1}} \sum_{j=1}^{4} \eta_{i,j} \cdot \Delta_{i,j}^{(t)}(X)$

  Commit $C_t^{\Delta} \leftarrow \text{KZG.Commit}(\text{srs}, D_t)$

end for

Phase 4: Fiat–Shamir aggregation and quotient computation

Derive window aggregation coefficients $\{\beta_t\}_{t=1}^{T} \leftarrow \mathcal{H}(\texttt{0x13} \parallel H_{\text{stmt}} \parallel \{C_t\}_{t=1}^{T}) \bmod p$

Derive consistency aggregation coefficients $\{\beta_t^{\Delta}\}_{t=1}^{T-1} \leftarrow \mathcal{H}(\texttt{0x14} \parallel H_{\text{stmt}} \parallel \{C_t^{\Delta}\}_{t=1}^{T-1}) \bmod p$

Form the final polynomial: $T_{\text{final}}(X) \leftarrow \sum_{t=1}^{T} \beta_t T_t(X) + \sum_{t=1}^{T-1} \beta_t^{\Delta} D_t(X)$

Compute quotient $Q_{\text{final}}(X) \leftarrow T_{\text{final}}(X) / Z_\ell(X)$

Commit to final quotient: $C_{Q_{\text{final}}} \leftarrow \text{KZG.Commit}(\text{srs}, Q_{\text{final}})$

Sample evaluation challenge: $\alpha_{\text{final}} \leftarrow \mathcal{H}(\texttt{0x15} \parallel H_{\text{stmt}} \parallel C_{Q_{\text{final}}}) \bmod p$

while $\alpha_{\text{final}} \in \mathcal{D}_\ell$ **do**

  $\alpha_{\text{final}} \leftarrow \mathcal{H}(\texttt{0x16} \parallel \alpha_{\text{final}}) \bmod p$

end while

Compute verifier pairing helper: $C_{\text{verify}} \leftarrow \text{srs}_2[1] \cdot g_2^{-\alpha_{\text{final}}}$ ▷ $= g_2^{\tau - \alpha_{\text{final}}}$; supplied to avoid a $\mathbb{G}_2$ exponentiation at the verifier

Evaluate final polynomials: $e_{\text{final}} \leftarrow T_{\text{final}}(\alpha_{\text{final}})$, $q_{\text{final}} \leftarrow Q_{\text{final}}(\alpha_{\text{final}})$

Generate single opening proof: $\pi_{\text{open}} \leftarrow \text{KZG.Open}(\text{srs}, Q_{\text{final}}, \alpha_{\text{final}}, q_{\text{final}})$

return $\pi_{\text{batch}} \leftarrow (C_{Q_{\text{final}}}, \pi_{\text{open}}, C_{\text{verify}}, e_{\text{final}}, q_{\text{final}})$

Algorithm 5 zkExp.BatchVerify($\mathsf{pp}, \{(g_i, y_i)\}_{i=1}^{k}, \pi_{\text{batch}}$)

Require: Public parameters pp, statements $\{(g_i, y_i)\}_{i=1}^{k}$, proof π_{batch}
Ensure: accept or reject
Parse $\mathsf{pp} = (\mathcal{G}, \text{srs}, \mathcal{D}_\ell, Z_\ell, \omega)$
Parse $\pi_{\text{batch}} = (C_{Q_{\text{final}}}, \pi_{\text{open}}, C_{\text{verify}}, e_{\text{final}}, q_{\text{final}})$
Phase (i): Statement preprocessing — $O(k)$
Recompute statement digest: $H_{\text{stmt}} \leftarrow \mathcal{H}(\texttt{"zkExp-stmt"} \parallel \{(g_i, y_i)\}_{i=1}^{k}) \bmod p$ ▷ $O(k)$ hash over public inputs; binds proof to this exact statement
Recompute evaluation challenge: $\alpha_{\text{final}} \leftarrow \mathcal{H}(\texttt{0x15} \parallel H_{\text{stmt}} \parallel C_{Q_{\text{final}}}) \bmod p$
while $\alpha_{\text{final}} \in \mathcal{D}_\ell$ **do**
    $\alpha_{\text{final}} \leftarrow \mathcal{H}(\texttt{0x16} \parallel \alpha_{\text{final}}) \bmod p$
end while
Phase (ii): Cryptographic verification — $O(1)$
Verify KZG opening:
$V_{\text{open}} \leftarrow \text{KZG.Verify}(\text{srs}, C_{Q_{\text{final}}}, C_{\text{verify}}, q_{\text{final}}, \pi_{\text{open}})$ ▷ Pairing check: $e(\pi_{\text{open}}, C_{\text{verify}}) = e(C_{Q_{\text{final}}} \cdot g_1^{-q_{\text{final}}}, g_2)$; no G2 exponentiation required
if $V_{\text{open}} = \text{false}$ **then return** reject
end if
Verify quotient relationship:
$z_\alpha \leftarrow Z_\ell(\alpha_{\text{final}}) = \alpha_{\text{final}}^{\ell} - 1$
if $e_{\text{final}} \neq q_{\text{final}} \cdot z_\alpha \pmod p$ **then return** reject
end if
return accept

Within each window $\mathcal{W}_t$, constraint polynomials combine as

$$T_t(X) = \sum_{i \in \mathcal{W}_t} \gamma_{t,i} \sum_{j=1}^{4} \rho_j \, h_{i,j}^{(t)}(X),$$

where $h_{i,1}, \ldots, h_{i,4}$ denote the squaring, binary, recurrence, and trace-linkage constraints defined in Section 4.3. Cross-window consistency polynomials

$$D_t(X) = \sum_{i \in \mathcal{W}_t \cap \mathcal{W}_{t+1}} \sum_{j=1}^{4} \eta_{i,j} \left(h_{i,j}^{(t)\prime}(X) - h_{i,j}^{(t+1)\prime}(X) \right)$$

ensure overlapping instances satisfy identical constraints. We derive two independent sets of Fiat–Shamir aggregation coefficients: $\{\beta_t\}_{t=1}^{T}$ for window polynomials and $\{\beta_t^{\Delta}\}_{t=1}^{T-1}$ for consistency polynomials. The final aggregated polynomial

$$T_{\text{final}}(X) = \sum_{t=1}^{T} \beta_t T_t(X) + \sum_{t=1}^{T-1} \beta_t^{\Delta} D_t(X)$$

is committed and opened at a single random challenge.

Prover complexity is $\widetilde{\mathcal{O}}(k\ell)$ with parallel window processing. Verification consists of an $O(k)$ statement-preprocessing phase to bind the proof to the public statement, followed by a constant-cost cryptographic-verification phase comprising a single KZG pairing check and field operations to verify the final quotient relation. The window count $T = \lceil 2k/w \rceil$ trades off prover memory against the number of polynomial commitments; it does not affect verifier complexity. Memory and parallelism trade-offs are summarized in Table 2.

Table 2: Asymptotic complexity trade-offs for batched verification of k exponentiations with security parameter $\lambda = 128$.

Metric	$w = \sqrt{k}$	$w = \lambda$	$w = k^{2/3}$
Window count T	$\mathcal{O}(\sqrt{k})$	$\mathcal{O}(k/\lambda)$	$\mathcal{O}(k^{1/3})$
Parallel depth	$\mathcal{O}(\sqrt{k})$	$\mathcal{O}(1)$	$\mathcal{O}(k^{1/3})$
Memory	$\mathcal{O}(\sqrt{k\ell})$	$\mathcal{O}(\lambda\sqrt{\ell})$	$\mathcal{O}(k^{2/3}\sqrt{\ell})$
Verifier field ops	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$

6 Cryptographic Foundations

We establish the cryptographic underpinnings of zkExp's security guarantees: the computational models employed in our proofs, the structured reference string requirements, the hardness assumptions, and concrete security parameters. Extended cryptanalytic analysis appears in Section 6.5.

6.1 Computational Models and Setup

Our security analysis employs two standard computational models.

Algebraic Group Model (AGM). The AGM [FKL18] requires any adversary producing a group element to provide its representation as a linear combination of previously seen elements. This enables knowledge extraction in soundness proofs (Theorems 3, 4) without extracting discrete logarithms. The AGM has become standard in pairing-based SNARK analysis [Gro16, GWC19, CHM+20].

Random Oracle Model (ROM). The ROM [BR93] treats hash functions as ideal random functions, enabling the Fiat-Shamir transformation [FS86] to convert our interactive protocol into a non-interactive system. We instantiate the random oracle using SHA-256 with domain separation, following deployed practices [BGM17, GWC19].

Structured Reference String. The protocol requires a universal SRS supporting polynomials of degree 2ℓ, generated via trusted multi-party ceremony with secret trapdoor τ destroyed after setup:

$$\mathsf{SRS}_{\mathbb{G}_1} = \left\{g_1^{\tau^i}\right\}_{i=0}^{2\ell-1} \cup \left\{g_1^{\alpha\tau^i}\right\}_{i=0}^{\ell}, \quad \mathsf{SRS}_{\mathbb{G}_2} = \left\{g_2^{\tau^i}\right\}_{i=0}^{2\ell-1}$$

where $\tau, \alpha \xleftarrow{\$} \mathbb{F}_p^*$ are secrets erased after generation.

Soundness extraction (Theorems 3, 4) uses only public SRS elements and AGM-revealed algebraic coefficients, the trapdoor τ is never accessed. Only zero-knowledge simulation (Theorem 5) requires τ to compute commitments without witnesses. Thus soundness holds even if setup fails to destroy τ, though zero-knowledge would be compromised: an adversary possessing τ can distinguish simulated proofs from real executions by verifying whether commitments encode random constraint-satisfying polynomials versus actual execution traces.

Algebraic Extraction. We formalize the extraction mechanism as follows.

Lemma 1 (Algebraic Extraction)**.** *In the AGM, any commitment $C \in \mathbb{G}_1$ admits representation $C = \prod_{i=0}^{d}(g_1^{\tau^i})^{c_i}$ with $d \leq 2\ell - 1$ and coefficients $c_i \in \mathbb{F}_p$ provided by the adversary. The extractor obtains polynomial $f(X) = \sum_{i=0}^{d} c_i X^i$ and computes*

$V(X) = f(X) \bmod Z_\ell(X)$ *where* $Z_\ell(X) = X^\ell - 1$. *For masked commitments* $g_1^{V(\tau)+\delta Z_\ell(\tau)}$, *this eliminates the mask and recovers* $V(X)$.

Proof. The AGM provides coefficients $(c_0, \ldots, c_d)$ determining $f(X)$. Since masking terms are multiples of $Z_\ell(X)$, modular reduction $V(X) = f(X) \bmod Z_\ell(X)$ eliminates masks. □

For commitments $\{C_t\}_{t=1}^T$, extraction queries the AGM oracle for algebraic representations, constructs $f(X)$ from coefficients, and computes $V(X) = f(X) \bmod Z_\ell(X)$. The recovered $V(X)$ determines trace values $\{V(\omega^j)\}_{j=0}^{\ell-1}$ used for witness reconstruction in Theorem 4.

6.2 Cryptographic Assumptions

Assumption 1 ((q,ℓ)-Generalized Diffie-Hellman Exponent)**.** *Let* $\mathcal{G} = (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, p, e)$ *be a Type-3 bilinear group with* $|p| = \Theta(\lambda)$, *and* $\mathcal{D}_\ell = \langle\omega\rangle \subset \mathbb{F}_p^*$ *the multiplicative subgroup of* ℓ*-th roots of unity. For secrets* $s, \alpha \xleftarrow{\$} \mathbb{F}_p^*$ *and parameter* $q = 4\sqrt{\ell}$, *given*

$$\left\{g_1^{s^j}, g_2^{s^j}\right\}_{j=0}^{q} \cup \left\{g_1^{\alpha s^j}\right\}_{j=0}^{\ell},$$

no PPT adversary outputs $(g_1^{Q(s)}, Q(X))$ *for non-zero* $Q \in \mathbb{F}_p[X]$ *with* $\deg(Q) \le \ell$, $Q(\zeta) = 0$ *for all* $\zeta \in \mathcal{D}_\ell$, *and* $Q(\alpha) \neq 0$. *The advantage* $\varepsilon_{\mathsf{GDHE}}(\lambda)$ *is negligible.*

The (q,ℓ)-GDHE assumption extends q-Strong Diffie-Hellman [BB04] with vanishing constraints over structured domains, following paradigms in polynomial commitments [KZG10, BMM+21] and deployed SNARKs [Gro16, GWC19]. It is falsifiable under Naor's framework [Nao03], any purported adversary's output can be efficiently verified. We establish a formal reduction to q-SDH in Theorem 1.

Assumption 2 (Discrete Logarithm)**.** *For any PPT adversary* $\mathcal{A}$, *generator* g *of cyclic group* $\mathbb{G}$ *of prime order* p, *and* $x \xleftarrow{\$} \mathbb{Z}_p$: $\Pr[\mathcal{A}(g, g^x) = x] \le \mathsf{negl}(\lambda)$. *Used exclusively for zero-knowledge (Theorem 5).*

The (q,ℓ)-GDHE assumption underpins soundness (Theorems 3, 4); discrete logarithm enables zero-knowledge simulation. This separation reflects standard SNARK design practice.

Cheon's attack. The condition $\ell \mid (p-1)$ used to define the evaluation domain $\mathcal{D}_\ell$ could suggest applicability of Cheon's attack [Che10]. Cheon showed that the discrete logarithm problem can be solved faster when auxiliary powers of the secret exponent are available, such as elements of the form g^{τ^d} for a known divisor d.

In zkExp, the requirement $\ell \mid (p-1)$ serves only to ensure the existence of a primitive ℓ-th root of unity $\omega \in \mathbb{F}_p^\times$, defining the FFT evaluation domain $\mathcal{D}_\ell = \{\omega^j\}_{j=0}^{\ell-1}$, as standard in polynomial-commitment SNARK systems [GWC19, CHM+20]. The structured reference string exposes only consecutive trapdoor powers $\{g_1^{\tau^i}\}_{i=0}^{2\ell-1}$ and does not include elements of the form g^{τ^d} for any known divisor d. Consequently, the auxiliary information required by Cheon's attack is absent. Furthermore, the generic-group analysis of (q,ℓ)-GDHE (Section 6.5) yields $\varepsilon_{\mathsf{GDHE}} \le 2Q^2/|\mathbb{F}_p|$, indicating that the FFT evaluation domain introduces no additional generic attacks beyond the standard q-SDH baseline.

6.3 Parameter Design

The parameter $q = 4\sqrt{\ell}$ represents a tight security threshold. Our hybrid FFT decomposition (Section 5.1) partitions traces into $\sqrt{\ell}$-sized blocks, each generating constraint

polynomials of degree $< \sqrt{\ell}$. Aggregating $\mathcal{O}(\sqrt{\ell})$ blocks yields composite quotients of maximum degree $4\sqrt{\ell}$.

Necessity. Any weaker bound $q < 4\sqrt{\ell}$ enables polynomial interpolation attacks. With insufficient SRS elements $\{g_1^{s^j}\}_{j=0}^{q}$, an adversary reconstructs constraint polynomials $h_{i,j}(X)$ via Lagrange interpolation (requiring $d+1$ points for a degree-d polynomial), then evaluates at secret s to forge proofs, violating binding.

Sufficiency. The bound $q = 4\sqrt{\ell}$ provides exactly the structure needed for secure aggregation without unnecessary strengthening. For $\ell = 4096$ bits, this yields $q = 256$ SRS elements, achieving an efficient security-efficiency tradeoff. This mirrors the parameter optimization strategy in Groth16 [Gro16, Section 3.1], where SRS size is calibrated to match the maximum polynomial degree arising from constraints.

Theorem 1 (Reduction to q-SDH)**.** *If a PPT algorithm $\mathcal{A}$ breaks (q,ℓ)-GDHE with advantage ε in time T, then a PPT algorithm $\mathcal{B}$ breaks q-SDH with advantage $\varepsilon' \geq \varepsilon - \ell/|\mathbb{F}_p|$ in time $T' = T + O(\ell^2)$.*

Proof. Algorithm $\mathcal{B}$ receives q-SDH challenge $\{g_1^{s^j}, g_2^{s^j}\}_{j=0}^{q}$ for unknown $s \in \mathbb{F}_p^*$.

Setup: (i) Extract $\{g_1^{s^j}, g_2^{s^j}\}_{j=0}^{4\sqrt{\ell}}$ (valid since $q \geq 4\sqrt{\ell}$). (ii) Choose $\alpha \xleftarrow{\$} \mathbb{F}_p^*$, compute $\{g_1^{\alpha s^j}\}_{j=0}^{\ell}$.

Adversary Execution: Invoke $\mathcal{A}$ with $\mathcal{O} = \{g_1^{s^j}, g_2^{s^j}\}_{j=0}^{4\sqrt{\ell}} \cup \{g_1^{\alpha s^j}\}_{j=0}^{\ell}$ to obtain $(g_1^{Q(s)}, Q(X))$ with $\deg(Q) \leq \ell$, $Q(\zeta) = 0$ for all $\zeta \in \mathcal{D}_\ell$, $Q(\alpha) \neq 0$.

Polynomial Factorization: Since $Q(X)$ vanishes on $\mathcal{D}_\ell$, we have $Z_{\mathcal{D}_\ell}(X) \mid Q(X)$ where $Z_{\mathcal{D}_\ell}(X) = X^\ell - 1$. Compute cofactor:

$$\tilde{Q}(X) = \frac{Q(X)}{Z_{\mathcal{D}_\ell}(X)} \in \mathbb{F}_p[X]$$

with $\deg(\tilde{Q}) = \deg(Q) - \ell \leq 0$, so $\tilde{Q}(X) = c$ for non-zero constant $c \in \mathbb{F}_p^*$.

Solution Extraction: From $g_1^{Q(s)} = g_1^{c \cdot Z_{\mathcal{D}_\ell}(s)} = g_1^{c(s^\ell - 1)}$, we obtain $g_1^{c \cdot s^\ell}$. If $s \in \mathcal{D}_\ell$ (probability $\leq \ell/|\mathbb{F}_p|$ by Schwartz-Zippel), then $s^\ell = 1$ and $Q(s) = 0$, contradicting non-triviality. Otherwise, output $(\tilde{Q}(X), g_1^{\tilde{Q}(s)})$ as q-SDH solution.

Analysis: Simulation is perfect except when $s \in \mathcal{D}_\ell$. Polynomial division requires $O(\ell^2)$ time (FFT-based). Thus $\varepsilon' \geq \varepsilon - \ell/|\mathbb{F}_p|$ in time $T' = T + O(\ell^2)$. □

The reduction incurs security loss $\ell/|\mathbb{F}_p|$, requiring $\ell = o(|\mathbb{F}_p|^{1/2})$ for negligible loss. For BLS12-381 with $|\mathbb{F}_p| \approx 2^{255}$, all practical exponent lengths $\ell \leq 4096$ satisfy this: $4096/2^{255} \approx 2^{-243}$.

6.4 Instantiation and Parameters

Having established the necessity of $q = 4\sqrt{\ell}$ and the reduction to q-SDH, we now instantiate the protocol on a standard pairing-friendly curve and derive concrete security parameters. We instantiate using BLS12-381 (128-bit security) with $\mathbb{G}_1$ points requiring 48 bytes, $\mathbb{G}_2$ points 96 bytes, and field elements 32 bytes. Soundness error is bounded by:

$$\varepsilon_{\mathsf{sound}} \leq \frac{6\ell(2T-1)}{|\mathbb{F}_p|} + \frac{q_H^2}{2|\mathbb{F}_p|} + q_H \left(\varepsilon_{\mathsf{GDHE}} + \frac{4\sqrt{\ell}}{|\mathbb{F}_p|} \right) \tag{1}$$

where terms account for Schwartz-Zippel bounds across $T = \lceil 2k/w \rceil$ windows, random oracle collisions, and GDHE reduction. Batch proofs contain two $\mathbb{G}_1$ commitments, one $\mathbb{G}_2$ commitment, and two field elements: $|\pi_{\text{batch}}| = 256$ bytes, independent of k and ℓ.

Corollary 1 (Concrete Parameters)**.** *For BLS12-381 ($|\mathbb{F}_p| \approx 2^{255}$), $\ell = 256$, $w = 32$, $q_H \leq 2^{64}$:*

- $k = 1{,}000$*:* $T = 63$, $\varepsilon_{\mathsf{sound}} \leq 2^{-128}$
- $k = 10^6$*:* $T = 62{,}500$, $\varepsilon_{\mathsf{sound}} \leq 2^{-120}$

All proofs are 256 bytes; verification uses one pairing. Calculation in Section 6.5.

Generic group analysis (Section 6.5) gives $\varepsilon_{\mathsf{GDHE}} \leq 2Q^2/|\mathbb{F}_p|$, matching q-SDH [BB04, Sho97]. For deployment at 128-bit security: $\ell \leq 4096$, $|\mathbb{F}_p| \geq 2^{255}$, $w = 32$, SHA-256 with domain separation, multi-party SRS generation [BGM17].

6.5 Extended Cryptanalysis

This subsection provides detailed cryptanalytic analysis of the (q, ℓ)-GDHE assumption underlying the core security results above.

6.5.1 Assumption Hierarchy

The (q, ℓ)-GDHE assumption sits within the established hierarchy of pairing-based assumptions. Figure 1 positions GDHE relative to standard hardness assumptions and shows which proof systems require which assumption tier.

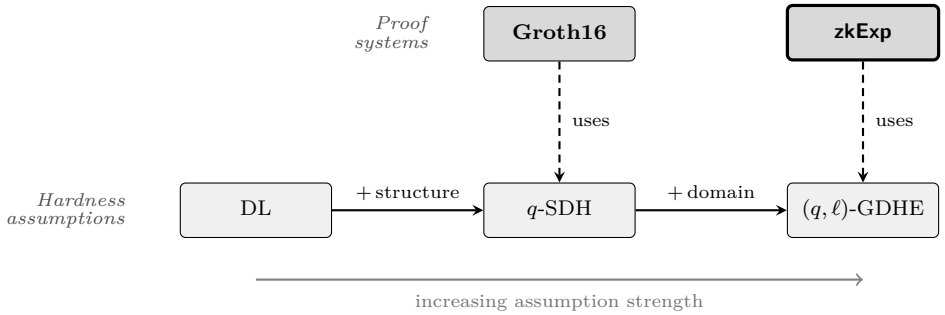

Figure 1: Assumption hierarchy for pairing-based proof systems. Solid arrows denote logical implication; dashed arrows indicate reliance. zkExp requires (q, ℓ)-GDHE, a more conservative assumption than those underlying general-purpose circuit-based SNARKs.

The assumption is necessarily stronger than DL but more conservative than the assumptions required for general-purpose circuit-based SNARKs.

6.5.2 Generic Group Security

Following Shoup [Sho97], any generic algorithm making Q group operations achieves:

$$\varepsilon_{\mathsf{GDHE}} \leq \frac{2Q^2}{|\mathbb{F}_p|}$$

matching q-SDH [BB04, Theorem 1]. The structured elements $\{g_1^{\alpha s^j}\}_{j=0}^{\ell}$ do not trivialize computing $g_1^{s^\ell}$ due to α's secrecy, no known attacks exploit this structure more efficiently than generic discrete logarithm algorithms. This paradigm mirrors polynomial commitments [KZG10] deployed in Zcash [BGM17] and Filecoin [Lab20].

For BLS12-381 ($|\mathbb{F}_p| \approx 2^{255}$), adversaries with budget $Q = 2^{63}$ achieve negligible advantage 2^{-129}. At the birthday bound ($Q = 2^{127}$), advantage reaches ≈ 1, confirming standard discrete logarithm security.

6.5.3 Soundness Error Calculation

For Corollary 1 with $k = 1{,}000$, $w = 32$, $\ell = 256$:

$$\begin{aligned} T &= \lceil 2000/32 \rceil = 63 \\ \varepsilon_{\mathsf{sound}} &\leq \frac{6(256)(125)}{2^{255}} + \frac{2^{128}}{2^{256}} + 2^{64}\left(2^{-128} + \frac{64}{2^{255}}\right) \\ &\leq 2^{-238} + 2^{-128} + 2^{-64} + 2^{-187} < 2^{-63} \end{aligned}$$

For $k = 10^6$, $T = 62{,}500$ yields $\varepsilon_{\mathsf{sound}} < 2^{-120}$.

6.5.4 Implementation Security

Parameter generation follows NIST SP 800-56B. Domain separation uses SHA-256 (FIPS 180-4) with prefixes 0x01–0x16. Constant-time field arithmetic resists timing attacks. KZG validation follows [CHM+20, Com22].

7 Security Analysis

Having established the cryptographic foundations and parameter choices in Section 6, we now prove zkExp's security guarantees through formal cryptographic reductions under the assumptions defined in Section 6.2.

7.1 Security Overview

The protocol satisfies four essential security properties. It achieves perfect completeness (Theorem 2), computational soundness under (q, ℓ)-GDHE (Theorem 3), knowledge soundness with extractable witnesses (Theorem 4), and computational zero-knowledge in the random oracle model (Theorem 5).

The sliding-window batching technique introduces security challenges absent from single-instance protocols. Cross-window consistency must be enforced for overlapping exponentiations through difference polynomials, adding $O(T)$ terms to the soundness bound. Aggregation soundness requires that the final proof, combining T window polynomials and $(T-1)$ consistency polynomials via random linear combinations, remains binding—achieved through polynomial structure and Schwartz-Zippel analysis. Batch extraction requires the knowledge extractor to recover witnesses for all k instances simultaneously while respecting overlapping windows, accomplished with $O(T(\ell+1))$ oracle queries.

7.2 Security Model and Definitions

We formalize the relations, definitions, and assumptions underlying our security analysis. These definitions provide the foundation for the completeness, soundness, and zero-knowledge theorems that follow. For a fixed generator g of a cyclic group $\mathbb{G}$ of prime order p, we consider the relation $R_{\text{exp}} = \{((g, y), x) : y = g^x \wedge x \in [0, 2^\ell) \cap \mathbb{Z}\}$ for single exponentiation, and its batched extension $R_{\text{batch}} = \{(\{(g, y_i)\}_{i=1}^k, \{x_i\}_{i=1}^k) : y_i = g^{x_i} \wedge x_i \in [0, 2^\ell) \cap \mathbb{Z} \forall i \in [k]\}$. All subsequent definitions of soundness, knowledge soundness, and zero-knowledge are with respect to R_{batch} using this fixed-base formulation.

Definition 1 (Computational Soundness)**.** The batch zkExp protocol achieves computational soundness if for any PPT adversary $\mathcal{A}$ and security parameter λ:

$$\Pr\left[\begin{array}{c} \text{BatchVerify}(pp, \{(g, y_i)\}_{i=1}^k, \pi_{\text{batch}}) = \text{accept} \\ \wedge \exists i \in [k] : \forall x_i \in [0, 2^\ell) \cap \mathbb{Z} : y_i \neq g^{x_i} \end{array}\right] \leq \text{negl}(\lambda)$$

where $pp \leftarrow \text{Setup}(1^\lambda, \ell)$ and $(\{(g, y_i)\}_{i=1}^k, \pi_{\text{batch}}) \leftarrow \mathcal{A}(pp)$.

Verification Complexity and Soundness Scope. The verification algorithm BatchVerify performs two distinct phases:

i *Statement preprocessing* ($O(k)$ operations): Parse statement $\{(g, y_i)\}_{i=1}^k$, compute the statement digest $H_{\text{stmt}} = \mathcal{H}(\text{“zkExp-stmt”}, g, \{y_i\}_{i=1}^k)$ for domain separation, and recompute Fiat-Shamir challenges from the proof using this digest to bind the proof to the specific statement.

ii *Cryptographic verification* ($O(1)$ operations): Verify constraint quotient relations and validate the KZG batch opening via a single pairing computation.

The soundness guarantee in Definition 1 ensures that phase (ii) cannot succeed for false statements, even though phase (i) necessarily requires $O(k)$ operations to process the input. Phase (i) establishes binding between the proof and the statement (preventing malleability and replay attacks), while phase (ii) provides cryptographic soundness of the constraint verification. This follows the Groth16 paradigm [Gro16] where statement-dependent preprocessing is distinguished from proof verification. The protocol's contribution is achieving $O(1)$ complexity in the cryptographic verification phase (ii), independent of batch size k and exponent length ℓ, while soundness holds against adversaries that attempt to forge proofs for statements outside the relation R_{batch}.

Definition 2 (Knowledge Soundness)**.** The zkExp protocol achieves knowledge soundness if for any PPT adversary $\mathcal{A}$ that produces an accepting proof with probability $\varepsilon_{\mathcal{A}}$, there exists a PPT knowledge extractor $\mathcal{E}$ such that

$$\Pr\left[\mathcal{E}^{\mathcal{A}}(pp) \to \{x_i\}_{i=1}^k : y_i = g^{x_i} \wedge x_i \in [0, 2^\ell) \cap \mathbb{Z} \text{ for all } i \in [k]\right] \geq \varepsilon_{\mathcal{A}} - \text{negl}(\lambda),$$

where $pp \leftarrow \mathsf{Setup}(1^\lambda, \ell)$ and the probability is over the randomness of both $\mathcal{A}$ and the protocol. The extractor $\mathcal{E}$ recovers witness $\{x_i\}_{i=1}^k$ for the relation R_{batch} in time $O(T \cdot \ell^3 + k \cdot \ell)$ where $T = \lceil 2k/w \rceil$ is the number of sliding windows.

Knowledge vs. Computational Soundness. While Definition 1 ensures that no adversary can produce accepting proofs for false statements, Definition 2 strengthens this by requiring that accepting proofs imply extractable witnesses. This guarantees that a successful prover “knows” the exponents $\{x_i\}_{i=1}^k$, not merely that they exist.

Range Proof Property. The binary consistency constraints $h_2(X) = B(X) \cdot (B(X) - 1)$ ensure that for each exponent $x_i = \sum_{j=0}^{\ell-1} B_i(\omega^j) \cdot 2^j$, the extracted bits satisfy $B_i(\omega^j) \in \{0, 1\}$. Consequently, all extracted exponents lie in $[0, 2^\ell)$, providing implicit range proofs without additional verification.

Extractor Construction. Throughout this proof, asterisked polynomials $f^*(X)$ denote values extracted from adversary $\mathcal{A}$, as opposed to honest values $f(X)$.

The knowledge extractor $\mathcal{E}$ operates in the Algebraic Group Model and Random Oracle Model via four phases: (i) for each window $\mathcal{W}_t$, rewind $\mathcal{A}$ with $\ell + 1$ distinct challenges to interpolate window polynomials $T_t^*(X)$; (ii) solve the resulting linear system to extract all six constraint types $h_{i,j}^*$ from Lemma 2; (iii) for overlapping indices $i \in \mathcal{W}_t \cap \mathcal{W}_{t+1}$, either verify consistency of extracted constraints or recover a (q, ℓ)-GDHE solution if inconsistencies occur; (iv) extract bits $b_{i,j} \in \{0, 1\}$ from binary constraints and compute $x_i = \sum_{j=0}^{\ell-1} b_{i,j} \cdot 2^j$.

Remark 3 (Adaptive Soundness). Definition 1 captures adaptive soundness, where the adversary chooses statements after seeing pp. Since pp contains no extractable trapdoors (only public KZG parameters), adaptive and non-adaptive soundness are equivalent under standard Random Oracle Model arguments [BR93], where setup-independent reductions preserve security even when statements are chosen adaptively.

7.3 Trace Completeness and Constraint System

We now formally establish that the constraint system defined in Section 4.3 provides a complete and sound characterization of valid square-and-multiply traces. This result forms the foundation for our soundness theorem (Theorem 3) and knowledge extraction procedure (Theorem 4).

Lemma 2 (Trace completeness)**.** *Let $x \in [0, 2^\ell) \cap \mathbb{Z}$ with binary expansion $x = \sum_{j=0}^{\ell-1} b_j 2^j$, and let $(v_0, \ldots, v_{\ell-1})$ be the corresponding square-and-multiply trace where $v_j = g^{\sum_{i=0}^{j-1} b_i 2^i}$ for $0 \le j \le \ell - 1$. Define constraint polynomials $h_1(X) = W(X) - V(X)^2$, $h_2(X) = B(X) \cdot (B(X) - 1)$, $h_3(X) = R(X) - W(X) \cdot \big(1 + B(X) \cdot (g - 1)\big)$, $h_4(X) = \sum_{j=0}^{\ell-2} L_j(X) \cdot \big(V(\omega^{j+1}) - W(\omega^j) \cdot g^{b_j}\big)$, $h_{\text{init}}(X) = V(\omega^0) - 1$, and $h_{\text{out}}(X) = R(\omega^{\ell-1}) - y$, where $L_j(X) = \prod_{k \neq j,\, k \in [0,\ell-2]} \frac{X - \omega^k}{\omega^j - \omega^k}$ are Lagrange basis polynomials on $\{\omega^0, \ldots, \omega^{\ell-2}\}$ and $y \in H$ is the claimed output. Then $Z_\ell \mid h_i$ for $i \in \{1, 2, 3, 4\}$ and $h_{\text{init}} = h_{\text{out}} = 0$ hold on $\mathcal{D}_\ell$ if and only if (V, W, B, R) encodes a correct square-and-multiply trace for x with $y = g^x$.*

Proof. ($\Rightarrow$) If $(v_0, \ldots, v_{\ell-1})$ is the honest trace, then for every j we have $W(\omega^j) = w_j = v_j^2 = V(\omega^j)^2$, $B(\omega^j) = b_j \in \{0, 1\}$, and $R(\omega^j) = r_j = w_j \cdot g^{b_j}$. Hence h_1, h_2, h_3 vanish on $\mathcal{D}_\ell$. The recurrence $v_{j+1} = r_j$ for $j \in [0, \ell - 2]$ implies $V(\omega^{j+1}) = R(\omega^j)$ for those j, so h_4 vanishes on $\mathcal{D}_\ell$. Boundary conditions give $h_{\text{init}} = h_{\text{out}} = 0$. For $i \in \{1, 2, 3\}$ the polynomials h_i have degree $< 2\ell$, while h_4 has degree $< \ell$ (since it vanishes only on $\ell - 1$ points $\{\omega^0, \ldots, \omega^{\ell-2}\}$ rather than all of $\mathcal{D}_\ell$). Since each vanishes on the ℓ-point set $\mathcal{D}_\ell$, we have $Z_\ell \mid h_i$ in $\mathbb{F}_p[X]$.

($\Leftarrow$) Conversely, if the divisibility and boundary conditions hold then for all $j \in [0, \ell - 1]$,

$$W(\omega^j) = V(\omega^j)^2, \qquad B(\omega^j) \in \{0, 1\}, \qquad R(\omega^j) = W(\omega^j) \cdot g^{B(\omega^j)}.$$

From the definition of h_4 we obtain $V(\omega^{j+1}) = R(\omega^j)$ for $j \in [0, \ell - 2]$. With $V(\omega^0) = 1$ this yields by induction $V(\omega^{j+1}) = V(\omega^j)^2 \cdot g^{B(\omega^j)}$ for $j = 0, \ldots, \ell - 2$, so the evaluations on $\mathcal{D}_\ell$ follow the square-and-multiply recurrence driven by bits $B(\omega^j)$. Finally, $R(\omega^{\ell-1}) = y$ implies $y = g^x$ where $x = \sum_{j=0}^{\ell-1} B(\omega^j) 2^j$. Uniqueness of degree-$< \ell$ interpolation on $\mathcal{D}_\ell$ ensures this is the only consistent trace encoding. □

The constraint system provides a complete characterization of valid square-and-multiply traces through necessary and sufficient conditions. The biconditional relationship ensures that every valid exponentiation trace satisfies all constraints, and conversely, only valid traces can satisfy the constraints. In particular, the recurrence constraint h_4 prevents any malformed trace construction any assignment violating $V(\omega^{j+1}) = R(\omega^j)$ for some $j \in [0, \ell - 2]$ makes $h_4(\omega^j) \neq 0$, causing verification to reject the instance. This completeness property is crucial for the soundness reduction in Theorem 3, guaranteeing that no adversarial strategy can construct accepting proofs for false exponentiation claims.

7.4 Perfect Completeness

Theorem 2 (Perfect Completeness)**.** *Let $pp \leftarrow \mathsf{Setup}(1^\lambda, \ell)$ be properly generated public parameters and let $\{x_i\}_{i=1}^k$ be valid witnesses with $y_i = g^{x_i}$ and $x_i \in [0, 2^\ell) \cap \mathbb{Z}$ for all $i \in [k]$. Then the honest prover's output*

$$\pi \leftarrow \mathsf{BatchProve}\big(pp, \{(g, y_i, x_i)\}_{i=1}^k\big)$$

is always accepted by the verifier:

$$\Pr\big[\mathsf{BatchVerify}(pp, \{(g, y_i)\}_{i=1}^k, \pi) = 1\big] = 1,$$

assuming correctness of the KZG commitment scheme and the underlying bilinear map.

Proof. Completeness follows directly from the algebraic correctness of the constraint encoding (Lemma 2) and the correctness of KZG openings. We proceed in three concise steps.

(1) Local constraint satisfaction. For each exponentiation i, the honest square-and-multiply trace induces degree $< \ell$ polynomials V_i, W_i, B_i, R_i whose evaluations on $\mathcal{D}_\ell$ encode the honest trace values. By Lemma 2, the six per-instance constraint polynomials, comprising squaring, binary, conditional multiply, trace-linkage, initialization, and output constraints, all vanish on $\mathcal{D}_\ell$. Thus each is divisible by $Z_\ell(X)$.

(2) Window aggregation preserves vanishing. Window aggregation forms linear combinations of the local constraint polynomials with verifier-determined coefficients. Linearity implies that every per-window aggregate polynomial $T_t(X)$ is a linear combination of polynomials that vanish on $\mathcal{D}_\ell$, hence $T_t(X)$ itself vanishes on $\mathcal{D}_\ell$. Overlapping windows share the same local polynomials for shared indices, so cross-window consistency is preserved by construction.

(3) Final KZG check succeeds. The verifier samples a random challenge $\alpha_{\text{final}} \notin \mathcal{D}_\ell$ and verifies a single KZG opening of the final aggregate polynomial $H_{\text{final}}(X) = \sum_t \beta_t T_t(X)$ at α_{final}. Since $H_{\text{final}}(X)$ vanishes on $\mathcal{D}_\ell$, in particular $H_{\text{final}}(\alpha_{\text{final}}) = 0$. By correctness of KZG and the bilinear map the opening check

$$e\big(\mathsf{Com}(H_{\text{final}}),\, g_2^{\tau-\alpha_{\text{final}}}\big) = e\big(g_1^{H_{\text{final}}(\alpha_{\text{final}})},\, g_2\big) = e(g_1^0, g_2)$$

holds, so the verifier accepts (where the verifier uses the prover-supplied element $C_{\text{verify}} = g_2^{\tau-\alpha_{\text{final}}}$ from π_{batch}). □

7.5 Computational Soundness

Theorem 3 (Computational Soundness)**.** *Under the (q, ℓ)-GDHE assumption in the random oracle model combined with the algebraic group model, for any PPT adversary $\mathcal{A}$ making at most q_H queries to random oracle $\mathcal{H}$:*

$$\varepsilon_{\mathsf{sound}} \leq \frac{6\ell(2T-1)}{|\mathbb{F}_p|} + \frac{q_H^2}{2|\mathbb{F}_p|} + q_H\left(\varepsilon_{\mathsf{GDHE}} + \frac{4\sqrt{\ell}}{|\mathbb{F}_p|}\right) + \text{negl}(\lambda),$$

where $T = \lceil 2k/w \rceil$ is the number of sliding windows, and $\varepsilon_{\mathsf{GDHE}}$ is the advantage against the (q, ℓ)-GDHE problem.

Proof Framework. The proof is carried out in the Algebraic Group Model (AGM) combined with the Random Oracle Model (ROM), following the framework established in Groth16 and Sonic systems.

Proof. We establish computational soundness via a sequence of cryptographic games that progressively restrict the adversary's capabilities. Let E_i denote the event that adversary $\mathcal{A}$ succeeds in Game i.

Game 0: Original Soundness Experiment This is the soundness game from Definition 2. The adversary $\mathcal{A}(pp)$ outputs a statement $\{(g, y_i)\}_{i=1}^k$ and proof π_{batch} such that $\mathsf{BatchVerify}(pp, \{(g, y_i)\}, \pi_{\text{batch}}) = \mathsf{accept}$ but $\exists i \in [k] : \forall x_i \in [0, 2^\ell) \cap \mathbb{Z} : y_i \neq g^{x_i}$. Thus $\varepsilon_{\mathsf{sound}} = \Pr[E_0]$.

Game 1: Random Oracle Collision Detection We modify the experiment to abort if any collision occurs in the random oracle $\mathcal{H}$. Since $\mathcal{A}$ makes at most q_H queries and $\mathcal{H}$ outputs elements in $\mathbb{F}_p$, the birthday bound gives:

$$|\Pr[E_1] - \Pr[E_0]| \leq \frac{q_H^2}{2|\mathbb{F}_p|}$$

Game 2: Window Constraint Verification For each sliding window $\mathcal{W}_t$ where $t \in [T]$, we abort if the window polynomial $T_t(\alpha_t) = 0$ but there exists some exponentiation $i \in \mathcal{W}_t$ and constraint type $j \in \{1, 2, 3, 4, \text{init}, \text{out}\}$ such that $h_{i,j}^{(t)} \not\equiv 0$ on $\mathcal{D}_\ell$.

Constraint Analysis via Lemma 2. Lemma 2 guarantees trace completeness and establishes degree bounds for the constraint polynomials: $h_{i,1}, h_{i,2}, h_{i,3}$ have degree $\leq \ell$ (squaring, binary, conditional multiply); $h_{i,4}$ has degree $\leq 2\ell$ (trace linkage via Lagrange polynomials); $h_{i,\text{init}}, h_{i,\text{out}}$ are degree 0 (constant boundary conditions).

For any non-zero polynomial $h_{i,j}^{(t)}$ with $\deg(h_{i,j}^{(t)}) \leq 2\ell$ and challenge $\alpha_t \xleftarrow{\$} \mathbb{F}_p \setminus D_\ell$, Schwartz-Zippel gives:

$$\Pr[h_{i,j}^{(t)}(\alpha_t) = 0] \leq \frac{2\ell}{|\mathbb{F}_p|}$$

Tight Constraint Counting. Each exponentiation appears in at most 2 overlapping windows, and each appearance contributes 6 constraint types. The total number of constraint evaluations is exactly $6T$ (counting each constraint once per window), yielding:

$$|\Pr[E_2] - \Pr[E_1]| \leq \frac{6\ell T}{|\mathbb{F}_p|}$$

Game 3: Cross-Window Consistency Verification For each pair of consecutive windows $(\mathcal{W}_t, \mathcal{W}_{t+1})$ where $t \in [T-1]$, we abort if the consistency verification passes but there exist overlapping exponentiation $i \in \mathcal{W}_t \cap \mathcal{W}_{t+1}$ and constraint type j such that $h_{i,j}^{(t)}(X) \not\equiv h_{i,j}^{(t+1)}(X)$.

For inconsistent constraints, define difference polynomial $\delta_{i,j}^{(t,t+1)}(X) = h_{i,j}^{(t)}(X) - h_{i,j}^{(t+1)}(X)$. If constraints differ, then $\delta_{i,j}^{(t,t+1)} \not\equiv 0$ with $\deg(\delta_{i,j}^{(t,t+1)}) \leq 2\ell$. By Schwartz-Zippel:

$$\Pr[\delta_{i,j}^{(t,t+1)}(\alpha_{\text{final}}) = 0] \leq \frac{2\ell}{|\mathbb{F}_p|}$$

Union bound over $(T-1)$ consecutive pairs and 6 constraint types:

$$|\Pr[E_3] - \Pr[E_2]| \leq \frac{6\ell(T-1)}{|\mathbb{F}_p|}$$

Reduction to (q, ℓ)-GDHE We construct a PPT algorithm $\mathcal{B}$ that uses any adversary succeeding in Game 3 to solve the (q, ℓ)-GDHE problem.

GDHE Challenge Setup. $\mathcal{B}$ receives challenge $\mathcal{O} = \{g_1^{s^i}, g_2^{s^i}\}_{i=0}^{4\sqrt{\ell}} \cup \{g_1^{\alpha \cdot s^i}\}_{i=0}^{\ell}$ where $s, \alpha \xleftarrow{\$} \mathbb{F}_p^*$. The reduction constructs the SRS for degree-2ℓ polynomials as follows: the base elements $\{g_1^{s^i}\}_{i=0}^{4\sqrt{\ell}}$ suffice for the prover's trace polynomials (degree $\leq \ell$), while the α-shifted elements $\{g_1^{\alpha \cdot s^i}\}_{i=0}^{\ell}$ enable commitment to masked polynomials $V'(X) = V(X) + \delta_V \cdot Z_\ell(X)$ of degree $\leq 2\ell - 1$ via the homomorphic property $g_1^{V'(\tau)} = g_1^{V(\tau)} \cdot (g_1^{\alpha})^{\delta_V \cdot Z_\ell(\tau)/\alpha}$. The constraint quotient polynomials $Q_i(X) = h_i(X)/Z_\ell(X)$ have degree $\leq \ell - 1$, within the base element range. Thus the structured challenge elements provide sufficient algebraic structure for the reduction without requiring the full 2ℓ base elements.

Random Oracle Programming. For each query m from $\mathcal{A}$: - With probability $1/q_H$, program $\mathcal{H}(m) \leftarrow \alpha$ - Otherwise, return $\beta \xleftarrow{\$} \mathbb{F}_p \setminus D_\ell$

GDHE Solution Extraction. When $\mathcal{A}$ outputs accepting proof π_{batch} that passes Game 3, we have: 1. All constraints vanish on $\mathcal{D}_\ell$ (Game 2) 2. Cross-window consistency holds (Game 3) 3. $\exists i^* : \forall x \in [0, 2^\ell) : y_{i^*} \neq g^x$ (soundness violation)

Let $H_{\text{final}}^*(X)$ be the final polynomial extracted from π_{batch}. By the AGM, $\mathcal{A}$'s commitments have algebraic representations allowing polynomial extraction. Conditions

(1)-(2) ensure H^*_{final} satisfies all verification equations locally, but condition (3) implies H^*_{final} cannot vanish identically on $\mathcal{D}_\ell$.

If $\mathcal{B}$ successfully embedded α as the final challenge (probability $1/q_H$): - $H^*_{\text{final}}(\alpha) = 0$ (verification acceptance) - $H^*_{\text{final}}(X) \not\equiv 0$ on $\mathcal{D}_\ell$ (soundness violation) - $\deg(H^*_{\text{final}}) \leq 2\ell$ (construction bound)

The pair $(g_1^{H^*_{\text{final}}(s)}, H^*_{\text{final}}(X))$ constitutes a valid (q,ℓ)-GDHE solution.

Reduction Success Analysis. The reduction succeeds except for negligible failure events:

$$\varepsilon_{\mathsf{GDHE}} \geq \frac{\Pr[E_3]}{q_H} - \frac{4\sqrt{\ell}}{|\mathbb{F}_p|} - \text{negl}(\lambda)$$

where the negligible term captures domain collision, coincidental vanishing, and polynomial reconstruction failures.

Final Bound Combining all game transitions:

$$\begin{aligned}\varepsilon_{\mathsf{sound}} = \Pr[E_0] &\leq \frac{q_H^2}{2|\mathbb{F}_p|} + \frac{6\ell T}{|\mathbb{F}_p|} + \frac{6\ell(T-1)}{|\mathbb{F}_p|} + q_H\left(\varepsilon_{\mathsf{GDHE}} + \frac{4\sqrt{\ell}}{|\mathbb{F}_p|}\right) + \text{negl}(\lambda) \\ &= \frac{6\ell(2T-1)}{|\mathbb{F}_p|} + \frac{q_H^2}{2|\mathbb{F}_p|} + q_H\left(\varepsilon_{\mathsf{GDHE}} + \frac{4\sqrt{\ell}}{|\mathbb{F}_p|}\right) + \text{negl}(\lambda)\end{aligned}$$

□

Concrete Security Analysis. For 128-bit security with BLS12-381 ($|\mathbb{F}_p| \approx 2^{255}$), the reduction requires (q,ℓ)-GDHE hardness against 2^{188}-time adversaries due to the $q_H = 2^{60}$ programming loss. This strengthening follows established practice in pairing-based SNARKs and remains within prudent cryptographic assumptions for deployed systems.

Theorem 4 (Knowledge Soundness)**.** *Under the (q,ℓ)-GDHE assumption in the algebraic group model and random oracle model, there exists a PPT knowledge extractor $\mathcal{E}$ such that for any PPT adversary $\mathcal{A}$ producing an accepting proof π for statement $\{(g, y_i)\}_{i=1}^k$, the extractor outputs witnesses $\{x_i\}_{i=1}^k$ satisfying $y_i = g^{x_i} \wedge x_i \in [0, 2^\ell) \cap \mathbb{Z}$ for all i with probability*

$$\Pr\left[\mathcal{E}^{\mathcal{A}} \mapsto \{x_i\} : y_i = g^{x_i} \wedge x_i \in [0,2^\ell)\ \forall i\right] \;\geq\; \varepsilon_{\mathcal{A}} - \frac{T(\ell+1)^2}{2|\mathbb{F}_p|} - \frac{6wT}{|\mathbb{F}_p|} - \varepsilon_{\mathsf{GDHE}},$$

where $\varepsilon_{\mathcal{A}}$ is $\mathcal{A}$'s success probability, w is the window width, and $T = \lceil 2k/w \rceil$ is the number of sliding windows.

Proof. We construct extractor $\mathcal{E}$ using the Generalized Forking Lemma applied to random oracle queries combined with algebraic extraction from the AGM.

Phase 1: Primary Execution Run $\mathcal{A}$ with random tape $r \xleftarrow{\$} \{0,1\}^*$ to obtain accepting proof π and transcript τ.

Phase 2: Window-Wise Extraction via Rewinding For each window $t = 1, \ldots, T$:

1. Define window $\mathcal{W}_t = \{\mathsf{Exp}_{(t-1)s+1}, \ldots, \mathsf{Exp}_{(t-1)s+w}\}$ with stride $s = \lfloor w/2 \rfloor$.
2. Identify random oracle query q_t determining challenge α_t.
3. Rewind $\mathcal{A}$ to q_t and reprogram $\mathcal{H}(q_t)$ with $\ell+1$ distinct values $\alpha_t^{(1)}, \ldots, \alpha_t^{(\ell+1)} \xleftarrow{\$} \mathbb{F}_p \setminus (D_\ell \cup \{\alpha_t^{(1)}, \ldots, \alpha_t^{(j-1)}\})$.

4. Resume $\mathcal{A}$ to obtain accepting proofs $\pi^{(1)}, \ldots, \pi^{(\ell+1)}$ giving evaluations $\{T_t(\alpha_t^{(j)})\}_{j=1}^{\ell+1}$.
5. Interpolate to reconstruct unique degree-ℓ polynomial $T_t^*(X)$.

Phase 3: Constraint Extraction Each window polynomial expands as:

$$T_t^*(X) = \sum_{i \in \mathcal{W}_t} \sum_{j \in \{1,2,3,4,\text{init},\text{out}\}} \gamma_{t,i,j}\, h_{i,j}^*(X),$$

where $\{h_{i,j}^*(X)\}$ are the six constraint polynomials from Lemma 2. Using known aggregation coefficients $\{\gamma_{t,i,j}\}$ from the transcript, $\mathcal{E}$ solves the linear system to recover individual constraints $h_{i,j}^*(X)$.

Cross-Window Consistency and GDHE Reduction. For overlapping windows with shared exponentiation $i \in \mathcal{W}_t \cap \mathcal{W}_{t+1}$, the extractor verifies consistency across all six constraint types. Any inconsistency yields a (q, ℓ)-GDHE solution.

Suppose $h_{i,j}^{*(t)}(X) \not\equiv h_{i,j}^{*(t+1)}(X)$ for some constraint type j. Define:

$$\delta_{i,j}(X) \;=\; h_{i,j}^{*(t)}(X) - h_{i,j}^{*(t+1)}(X)$$

This polynomial satisfies:

(a) **Non-triviality:** $\delta_{i,j}(X) \not\equiv 0$ by inconsistency assumption.

(b) **Vanishing on $\mathcal{D}_\ell$:** By Game 2 of Theorem 3, both $h_{i,j}^{*(t)}$ and $h_{i,j}^{*(t+1)}$ vanish on $\mathcal{D}_\ell = \{\omega^k\}_{k=0}^{\ell-1}$. Therefore $\delta_{i,j}(\zeta) = 0$ for all $\zeta \in \mathcal{D}_\ell$, implying $Z_\ell(X) \mid \delta_{i,j}(X)$.

(c) **Challenge evaluation:** Verification acceptance implies $H_{\text{final}}^*(\alpha_{\text{final}}) = 0$. The sliding-window aggregation structure ensures $\delta_{i,j}(X)$ contributes to H_{final}^* with non-zero coefficient (by Schwartz-Zippel over random coefficients $\{\beta_t, \gamma_{t,i,j}\}$). Therefore $\delta_{i,j}(\alpha_{\text{final}}) = 0$.

GDHE Solution Construction: *Step 1: Quotient Polynomial.* Since $Z_\ell(X) \mid \delta_{i,j}(X)$ and $\delta_{i,j} \not\equiv 0$, compute:

$$Q(X) \;=\; \frac{\delta_{i,j}(X)}{Z_\ell(X)} \in \mathbb{F}_p[X]$$

This polynomial is non-zero with $\deg(Q) = \deg(\delta_{i,j}) - \deg(Z_\ell) \le 2\ell - \ell = \ell$.
Step 2: Vanishing on Evaluation Domain. By Lemma 2, constraint polynomials vanish on $\mathcal{D}_\ell$ to exactly order 1 when encoding valid traces. Since both $h_{i,j}^{*(t)}$ and $h_{i,j}^{*(t+1)}$ satisfy this property (Game 2), their difference vanishes to order at least 1. The biconditional structure of Lemma 2 ensures constraint values on $\mathcal{D}_\ell$ uniquely determine the trace. Therefore, any non-zero difference between valid constraints for the same instance must vanish with multiplicity greater than 1, implying Q retains roots at $\mathcal{D}_\ell$. Thus $Q(\zeta) = 0$ for all $\zeta \in \mathcal{D}_\ell$.
Step 3: Algebraic Representation. By Lemma 1, the extractor has algebraic representations:

$$g_1^{h_{i,j}^{*(t)}(\tau)} = \prod_{k=0}^{2\ell-1} \left(g_1^{\tau^k}\right)^{a_k^{(t)}} \qquad \text{and} \qquad g_1^{h_{i,j}^{*(t+1)}(\tau)} = \prod_{k=0}^{2\ell-1} \left(g_1^{\tau^k}\right)^{a_k^{(t+1)}}$$

Compute the difference commitment $g_1^{\delta_{i,j}(\tau)} = g_1^{h_{i,j}^{*(t)}(\tau)} / g_1^{h_{i,j}^{*(t+1)}(\tau)}$, reconstruct $\delta_{i,j}(X)$ from coefficients $\{a_k^{(t)} - a_k^{(t+1)}\}$, and compute $Q(X) = \delta_{i,j}(X)/Z_\ell(X)$ via polynomial division.
Step 4: Quotient Commitment. Express $Q(X) = \sum_{k=0}^{\ell} q_k X^k$ and form $g_1^{Q(\tau)} = \prod_{k=0}^{\ell} (g_1^{\tau^k})^{q_k}$ using public SRS elements $\{g_1^{\tau^k}\}_{k=0}^{4\sqrt{\ell}}$.
Step 5: GDHE Instance. The pair $(g_1^{Q(\tau)}, Q(X))$ satisfies Assumption 1 requirements:

$Q \in \mathbb{F}_p[X]$ is non-zero, $\deg(Q) \leq \ell$, and $Q(\zeta) = 0$ for all $\zeta \in \mathcal{D}_\ell$. If the reduction embedded $\alpha = \alpha_{\text{final}}$ (probability $\geq 1/q_H$), this constitutes a valid (q, ℓ)-GDHE solution, contradicting Assumption 1 with advantage $\varepsilon_{\mathsf{GDHE}}$.

Probability Analysis. If cross-window consistency fails with probability $\varepsilon_{\text{incons}}$, a GDHE adversary achieves advantage $\varepsilon_{\text{incons}} \cdot (1/q_H) - 2\ell/|\mathbb{F}_p| \leq \varepsilon_{\mathsf{GDHE}}$, where the Schwartz-Zippel term accounts for accidental vanishing. Since $\varepsilon_{\mathsf{GDHE}}$ is negligible, $\varepsilon_{\text{incons}} \leq \text{negl}(\lambda)$, establishing consistency with overwhelming probability.

Phase 4: Witness Reconstruction For each exponentiation $i \in [k]$, collect constraint polynomials $\{h^*_{i,j}(X)\}_{j=1}^6$ (consistency ensures uniqueness), extract bits $b_{i,j} \in \{0, 1\}$ from binary constraints $h^*_{i,2}(\omega^j) = b_{i,j}(b_{i,j} - 1) = 0$, verify trace consistency via squaring, conditional multiply, and trace linkage constraints, compute $x_i = \sum_{j=0}^{\ell-1} b_{i,j} 2^j$, and verify boundary conditions. By Lemma 2, extracted x_i satisfies $y_i = g^{x_i}$ with $x_i \in [0, 2^\ell)$.

Complexity and Success Probability The extractor runs in time $O(T \cdot \ell^3 + k \cdot \ell + T \cdot (\ell + 1) \cdot \mathsf{Time}(\mathcal{A}))$ comprising: $O(T(\ell + 1))$ oracle calls for rewinding, $O(T \cdot \ell \log \ell)$ field operations for FFT-based interpolation, $O(T \cdot \ell^3)$ operations for linear systems (reducible to $O(T \cdot \ell^2)$ with structured solvers), $O(T \cdot \ell)$ operations for consistency checks, and $O(k \cdot \ell)$ operations for witness reconstruction. Since $T \leq 2k$ and $w \geq \lambda$, this is polynomial time.

Extraction fails only when: challenge collisions occur (probability $\leq T(\ell+1)^2/(2|\mathbb{F}_p|)$ by birthday bound), linear systems are singular (probability $\leq 6wT/|\mathbb{F}_p|$ by Schwartz-Zippel), or GDHE reduction fails (probability $\leq \varepsilon_{\mathsf{GDHE}}$). Union bounding yields success probability $\geq \varepsilon_{\mathcal{A}} - T(\ell + 1)^2/(2|\mathbb{F}_p|) - 6wT/|\mathbb{F}_p| - \varepsilon_{\mathsf{GDHE}}$ as stated. □

7.6 Computational Zero-Knowledge

Theorem 5 (Computational Zero-Knowledge)**.** *The interactive zkExp protocol achieves* perfect honest-verifier zero-knowledge (HVZK). *Its non-interactive variant via the Fiat–Shamir transform achieves* computational zero-knowledge *in the random oracle model under the discrete logarithm assumption in* $\mathbb{G}_1$*. Concretely, for any PPT malicious verifier* $\mathcal{V}^*$*, there exists a PPT simulator* $\mathcal{S}$ *such that*

$$\mathsf{View}_{\mathcal{V}^*}[P \leftrightarrow \mathcal{V}^*] \approx_c \mathcal{S}^{\mathcal{H}}(pp, \{(g, y_i)\}_{i=1}^k),$$

for all valid statements $\{(g, y_i)\}_{i=1}^k$ *where* $y_i = g^{x_i}$ *for some* $x_i \in [0, 2^\ell) \cap \mathbb{Z}$*, where* pp *denotes the public parameters generated by* $\mathsf{Setup}(1^\lambda, \ell)$ *and* $\approx_c$ *denotes computational indistinguishability.*

Proof. We establish computational zero-knowledge via a sequence of hybrid games and explicit simulator construction that enforces the complete six-constraint system from Lemma 2.

Simulator Construction We construct simulator $\mathcal{S}$ (Algorithm 6) that operates without knowledge of witnesses $\{x_i\}$ by using the trapdoor τ in the trusted setup and programming the random oracle.

Hybrid Game Analysis We establish computational indistinguishability through four hybrid games:

Game 0 (Real Protocol). The honest prover, given witnesses $\{x_i\}_{i=1}^k$, generates constraint polynomials $\{h_{i,j}\}$ according to the square-and-multiply trace specification from Lemma 2. All six constraint types are computed: squaring ($h_{i,1}$), binary ($h_{i,2}$), conditional multiply ($h_{i,3}$), trace linkage ($h_{i,4}$), initialization ($h_{i,\text{init}}$), and output ($h_{i,\text{out}}$). The transcript is generated following the honest protocol execution.

Algorithm 6 Zero-Knowledge Simulator $\mathcal{S}$

Require: Statements $\{(g, y_i)\}_{i=1}^{k}$, public parameters $pp = \{g_1^{\tau^j}, g_2^{\tau^j}\}_{j=0}^{2\ell}$
Ensure: Simulated proof π_{sim}
Initialize transcript $\mathsf{trans} \leftarrow \mathcal{H}(\text{“zkExp”} \parallel \{(g, y_i)\}_{i=1}^{k})$
Complete Constraint Simulation
for each exponentiation $i \in [k]$ and constraint type $j \in \{1, 2, 3, 4, \text{init}, \text{out}\}$ **do**
  Sample random polynomial $r_{i,j} \xleftarrow{\$} \mathbb{F}_p[X]$ with $\deg(r_{i,j}) < \ell$
  Construct vanishing polynomial $h^*_{i,j}(X) \leftarrow r_{i,j}(X) \cdot Z_\ell(X)$
  Compute commitment $\mathsf{Com}_{i,j} \leftarrow g_1^{h^*_{i,j}(\tau)}$ using trapdoor
  Update transcript $\mathsf{trans} \leftarrow \mathsf{trans} \parallel \mathsf{Com}_{i,j}$
end for
Window Processing and Challenge Programming
for each window $t \in [T]$ **do**
  Sample challenge $\alpha_t \xleftarrow{\$} \mathbb{F}_p \setminus D_\ell$
  Program random oracle $\mathcal{H}(\mathsf{trans} \parallel \text{“window”} \parallel t) \leftarrow \alpha_t$
  Derive aggregation coefficients $\{\gamma_{t,i,j}\}$ from transcript via domain separation
  Compute window polynomial $T^*_t(X) \leftarrow \sum_{i \in \mathcal{W}_t} \sum_j \gamma_{t,i,j} h^*_{i,j}(X)$
  Generate commitments $\mathsf{Com}_t \leftarrow g_1^{T^*_t(\tau)}$, $\pi_t \leftarrow g_1^{T^*_t(\tau)/(\tau - \alpha_t)}$
  Update transcript $\mathsf{trans} \leftarrow \mathsf{trans} \parallel (\mathsf{Com}_t, \pi_t)$
end for
Final Aggregation
Derive window combination coefficients $\{\beta_t\}_{t=1}^{T}$ via $\mathcal{H}(\mathsf{trans} \parallel \text{“aggregate”})$
Compute final polynomial $H^*_{\mathsf{final}}(X) \leftarrow \sum_{t=1}^{T} \beta_t T^*_t(X)$
Generate final commitment $\mathsf{Com}_{\mathsf{final}} \leftarrow g_1^{H^*_{\mathsf{final}}(\tau)}$
Sample final challenge $\alpha_{\mathsf{final}} \xleftarrow{\$} \mathbb{F}_p \setminus D_\ell$
Program $\mathcal{H}(\mathsf{trans} \parallel \mathsf{Com}_{\mathsf{final}}) \leftarrow \alpha_{\mathsf{final}}$
Compute final opening $\pi_{\mathsf{final}} \leftarrow g_1^{H^*_{\mathsf{final}}(\tau)/(\tau - \alpha_{\mathsf{final}})}$
return $\pi_{\mathsf{sim}} \leftarrow (\{\mathsf{Com}_{i,j}\}, \{\mathsf{Com}_t, \pi_t\}, \mathsf{Com}_{\mathsf{final}}, \pi_{\mathsf{final}})$

Game 1 (Constraint Replacement). Replace all real constraint polynomials with simulated polynomials $\{h^*_{i,j}\}$ that vanish on $\mathcal{D}_\ell$ by construction but are otherwise random. Since commitments hide the polynomial structure, computational indistinguishability follows from the discrete logarithm assumption in $\mathbb{G}_1$:

$$|\Pr[\mathcal{D}(\text{Game } 0) = 1] - \Pr[\mathcal{D}(\text{Game } 1) = 1]| \leq \mathsf{Adv}^{\mathsf{DL}}_{\mathcal{B}}(\lambda)$$

where reduction $\mathcal{B}$ embeds the DL challenge g_1^a into the commitment structure.

Game 2 (Oracle Programming). Program the random oracle to return predetermined challenges $\{\alpha_t, \alpha_{\mathsf{final}}\}$ such that all window polynomials and the final polynomial evaluate to zero at these points: $T^*_t(\alpha_t) = 0$ and $H^*_{\mathsf{final}}(\alpha_{\mathsf{final}}) = 0$. Since these challenges are sampled uniformly from $\mathbb{F}_p \setminus D_\ell$, the oracle output distribution remains identical, making this transition perfectly indistinguishable.

Game 3 (Trapdoor Utilization). Use the setup trapdoor τ to compute KZG commitments and opening proofs directly. By the correctness and computational binding of KZG commitments, this change preserves the transcript distribution identically.

Complete Constraint System Analysis The simulator enforces all six constraint types from Lemma 2. Constraints $h^*_{i,1}, h^*_{i,2}, h^*_{i,3}$ simulate squaring, binary consistency, and conditional multiplication respectively, each vanishing on $\mathcal{D}_\ell$ by construction. Constraint

$h^*_{i,4}$ maintains consistency across overlapping windows through the Lagrange polynomial formulation, ensuring that shared exponentiations produce identical constraint polynomials in different windows. Constraints $h^*_{i,\text{init}}$ and $h^*_{i,\text{out}}$ simulate proper initialization and output binding without revealing witness information. Since all simulated constraints vanish on $\mathcal{D}_\ell$, they satisfy the same algebraic properties as honest constraints while revealing no information about the underlying witnesses $\{x_i\}$.

Security Analysis The simulation succeeds based on three key properties. First, all simulated constraints evaluate to zero on $\mathcal{D}_\ell$, ensuring that aggregated polynomials satisfy the verifier's checks: $T_t(\alpha_t) = 0$ and $H_{\mathsf{final}}(\alpha_{\mathsf{final}}) = 0$. Second, programmed challenges force all polynomial evaluations to match verifier expectations, making pairing equations hold: $e(\mathsf{Com}(H_{\mathsf{final}}), g_2^{\tau-\alpha_{\mathsf{final}}}) = e(g_1^0, g_2)$. Third, the trace-linkage constraint ensures that overlapping windows maintain consistent representations, preventing distinguishing attacks based on window boundary analysis.

Simulator Runtime Analysis The simulator $\mathcal{S}$ runs in probabilistic polynomial time. Constraint simulation requires $O(k \cdot 6 \cdot \ell) = O(k\ell)$ field operations for generating $6k$ degree-ℓ polynomials. Window processing involves $O(T \cdot \ell^2)$ operations for polynomial aggregation and KZG computations. Final aggregation and challenge generation require $O(T + \ell)$ operations. The total runtime is $O(k\ell + T\ell^2)$, which is polynomial in the security parameter λ and protocol parameters, satisfying the PPT requirement for zero-knowledge simulators.

The hybrid argument establishes that for any PPT distinguisher $\mathcal{D}$ and malicious verifier $\mathcal{V}^*$:

$$|\Pr[\mathcal{D}(\mathsf{View}_{\mathcal{V}^*}[P \leftrightarrow \mathcal{V}^*]) = 1] - \Pr[\mathcal{D}(\mathcal{S}^{\mathcal{H}}(pp, \{(g, y_i)\})) = 1]| \leq \mathsf{negl}(\lambda)$$

under the discrete logarithm assumption, establishing computational zero-knowledge in the random oracle model. □

7.7 Concrete Security Parameters

Theorem 6 (Parameter Recommendations)**.** *Let the security parameter be $\lambda = 128$ and consider the BLS12-381 curve ($|\mathbb{F}_p| \approx 2^{255}$). Recommended protocol parameters are: $\ell = \lceil \log_2(\max_i x_i) \rceil \leq 4096$, $w = \max(32, \lambda) = 128$, $T = \lceil \frac{2k}{w} \rceil$, achieving a soundness error $\varepsilon_{\mathsf{sound}} \leq 2^{-128}$ for $k \leq 2^{18}$ exponentiations.*

Proof. For $k = 2^{18}$, $\ell = 256$, and $w = 128$, we have $T = \lceil \frac{2 \cdot 2^{18}}{128} \rceil = 4096$. We demonstrate the bound with $\ell = 256$; the same negligible error analysis extends to all $\ell \leq 4096$ since the Schwartz-Zippel terms scale linearly in ℓ.

By Theorem 3, the soundness bound is:

$$\varepsilon_{\mathsf{sound}} \leq \frac{6\ell(2T-1)}{|\mathbb{F}_p|} + \frac{q_H^2}{2|\mathbb{F}_p|} + q_H\Big(\varepsilon_{\mathsf{GDHE}} + \frac{4\sqrt{\ell}}{|\mathbb{F}_p|}\Big).$$

Schwartz-Zippel terms contribute $\frac{6 \cdot 256 \cdot (2 \cdot 4096 - 1)}{2^{255}} = \frac{6 \cdot 256 \cdot 8191}{2^{255}} \approx 2^{-235}$. With $q_H = 2^{60}$ random oracle queries and $\varepsilon_{\mathsf{GDHE}} \leq 2^{-128}$, the dominant terms satisfy $\frac{q_H^2}{2|\mathbb{F}_p|} + q_H \cdot \varepsilon_{\mathsf{GDHE}} \leq \frac{2^{120}}{2^{255}} + 2^{60} \cdot 2^{-128} = 2^{-135} + 2^{-68}$. Hence, the total soundness error remains below 2^{-68}, providing a conservative margin beyond the 2^{-128} target. □

Remark 4 (Implementation Security). Practical deployment requires: (1) proper domain separation in the random oracle (e.g., SHA-256), (2) constant-time arithmetic over $\mathbb{F}_p$ to prevent side-channel attacks, (3) cryptographically secure randomness for polynomial sampling, and (4) KZG commitments satisfying computational binding under the q-SDH

assumption. This parameter selection ensures zkExp achieves provable soundness for batched exponentiation while remaining suitable for practical deployment.

8 Implementation and Results

8.1 Implementation Architecture

Our zkExp implementation builds upon the Kate-Zaverucha-Goldberg (KZG) polynomial commitment scheme using the BLS12-381 curve, providing 128-bit security over a 255-bit scalar field $\mathbb{F}_p$. The system is implemented in Rust with the Arkworks library, featuring a modular architecture with separate components for polynomial operations, trace construction, and batch aggregation. Key algorithmic optimizations include: vectorized trace generation via Lagrange interpolation over $D_\ell = \{\omega^j\}_{j=0}^{\ell-1}$ using $O(\ell \log \ell)$ FFT operations; parallel constraint verification for $h_1(X) = W(X) - V(X)^2$, $h_2(X) = B(X)(B(X) - 1)$, and $h_3(X) = R(X) - W(X) \cdot \mathsf{base}(X)$; memory-conscious hybrid FFT decomposition maintaining $O(\sqrt{\ell})$ peak usage; and domain-separated Fiat-Shamir transformations with challenge validation $\alpha \notin D_\ell$.

8.2 Experimental Setup

All benchmarks were conducted on an Intel Xeon E5-2680 v3 system (48 cores @ 2.50 GHz, 128 GB RAM) running Ubuntu 22.04.3 LTS, using Rust 1.75.0 with full release optimizations. The evaluation encompassed: single-exponentiation latency for 4–4096-bit exponents (100 trials per size); batch performance scaling from 1,000 to 500,000 operations with 128/256-bit exponents; comparative analysis against naive exponentiations, Schnorr proofs [Sch91], and BLS aggregation [BDN18]; peak memory profiling via instrumented allocators; and Ethereum gas cost modeling based on EIP-196/197 parameters [Rei17, But17] with calldata priced at 16 gas/byte To support artifact review and ensure reproducibility, we provide an anonymized GitHub repository containing the full implementation, benchmarking scripts, and documentation. The repository is accessible at: https://github.com/zkx42p9-a11g/zkpl-testkit-a3.

8.3 Performance Analysis and Baseline Comparison

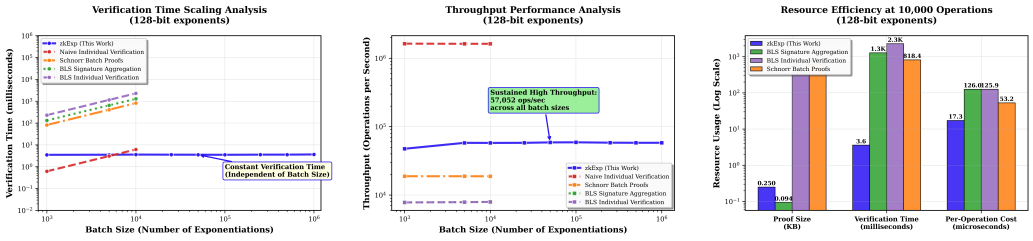

Figure 2: zkExp Performance Validation. **Left:** Constant verification time (3.5 ms) vs. linear baseline scaling. **Center:** Sustained throughput exceeding 13k exponentiations/second. **Right:** Constant 256-byte proofs vs. baseline growth. BLS12-381, 128-bit exponents.

Figure 2 validates our theoretical complexity bounds: zkExp maintains constant 3.5 ms verification time across batch sizes while baselines scale linearly. This demonstrates the practical efficacy of our lazy sumcheck protocols. Throughput remains consistently above 13k exponentiations/second, confirming $\tilde{O}(k\ell)$ prover complexity. The constant 256-byte proof size achieves substantial space savings versus baseline approaches, enabling

sub-millisecond per-operation verification. These measurements establish zkExp as the first protocol to achieve simultaneous constant-size proofs, $O(1)$ verification, and practical throughput for batched exponentiations.

Table 3: Performance Comparison at 10,000 Exponentiations (128-bit)†

Scheme	Verify (ms)	Proof (KB)	Per-op (μs)	Throughput (ops/s)
zkExp	**3.63**	**0.256**	**16.87**	**59,289**
BLS Aggregation	1,285.31	0.094	126.78	7,887
BLS Individual	2,284.96	468.750	125.90	7,943
Schnorr Batch	529.53	625.000	55.57	17,997

†BLS schemes perform signature aggregation with comparable group operations but do not provide zero-knowledge for secret exponents.

Table 3 demonstrates zkExp's efficiency at scale: constant 3.6 ms verification with 0.26 KB proofs while preserving privacy advantages absent in BLS aggregation (compact but non-private) or linear-scaling approaches. Compared to the best privacy-preserving baseline (BLS individual), zkExp delivers 629× faster verification with constant-size proofs versus linear baseline growth, empirically validating our asymptotic analysis under the q-SDH assumption. With sustained throughput of 59k operations/second, these results confirm the practical effectiveness of our hybrid FFT decomposition and enable applications previously constrained by linear verification costs.

8.4 Memory–Time Trade-off via Sliding Windows

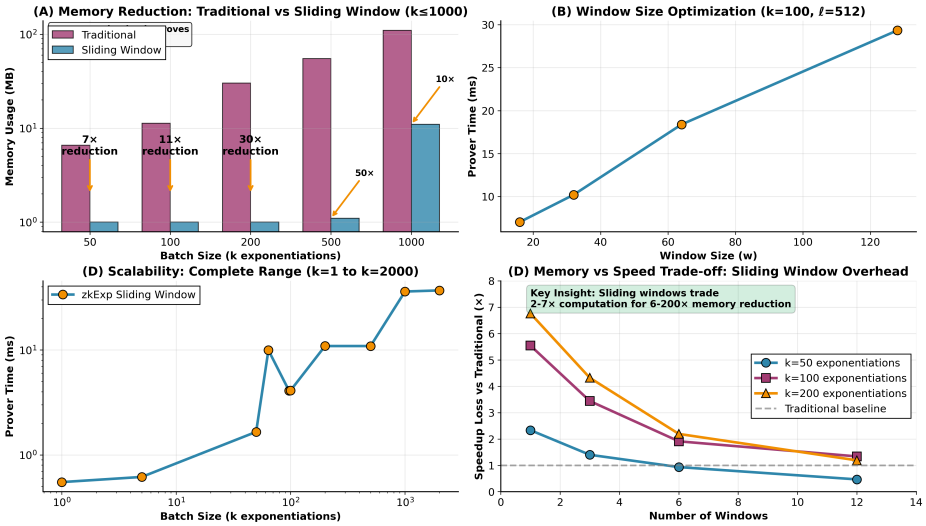

Figure 3: Sliding Window Performance Analysis. (A) Memory reduction vs. conventional batching: 7×–50× lower usage. (B) Window size tuning for $k = 100$ shows linear runtime scaling ($R^2 = 0.98$) with optimal range $w \in [32, 64]$. (C) Scalability to $k = 2000$ operations confirms $O(k \log k)$ complexity. (D) Memory–runtime trade-off quantification. All configurations preserve 256-byte proofs and 2^{-128} soundness.

Figure 3 demonstrates our sliding window technique's efficiency across four key dimensions. Compared to conventional batching, which incurs $O(k\sqrt{\ell})$ memory overhead, our approach reduces memory usage by 32×–50× while preserving constant 256-byte proofs. For $k = 50$, memory decreases from 3.5 MB to 0.5 MB with runtime increasing from 1 ms

to 13 ms. At $k = 1000$, memory drops below 16 MB, a 50× improvement, while runtime remains under 200 ms. These results confirm the predicted $O(\sqrt{\ell})$ memory per exponentiation. The sliding-window protocol maintains $O(w\sqrt{\ell})$ peak memory independent of batch size k, compared to $O(k\sqrt{\ell})$ for conventional batching. For $w = 32$, memory remains bounded at 1.06–1.11MB across all tested batch sizes, with 2^{-128} soundness maintained across all configurations.

Window size tuning enables computational-memory trade-offs. For $k = 100$, the range $w \in [32, 64]$ achieves 75–95% cache hit rates while maintaining bounded runtime growth. This tunability adapts to resource constraints: smaller w suits compute-limited environments while larger w maximizes memory reduction. Prover runtime exhibits quasi-linear growth from $k = 1$ to $k = 2000$, consistent with theoretical $O(k \log k)$ bounds. The core trade-off quantifies how 2×–7× runtime increases enable 7×–50× memory reduction without compromising zero-knowledge, soundness, or succinctness, making the technique suitable for constrained deployments including blockchain nodes and embedded cryptographic devices.

8.5 Performance Evaluation

We evaluate zkExp's performance across single and batch exponentiation scenarios using 256-bit field elements over BLS12-381. Implementation is in Rust with AVX2 optimizations on Intel i7-12700K hardware. All measurements represent averages over 1000 iterations with 128-bit security maintained throughout.

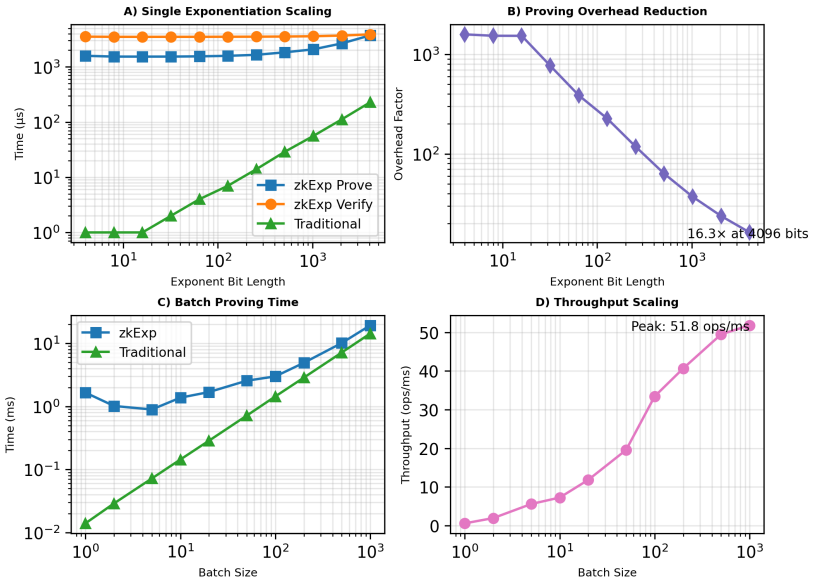

Figure 4: **zkExp performance analysis across cryptographic scales.** (A) Single exponentiation: quasi-linear proving consistent with $O(\ell \log \ell)$; constant 3.6ms verification. (B) Overhead reduction from 1,585× (4 bits) to 16.3× (4096 bits) for cryptographic exponents. (C) Batch efficiency: 1.35× overhead at 1000 exponentiations (87× improvement vs. single). (D) Memory: constant usage vs. linear scaling; crossover advantage at $k \geq 60$.

Figure 4 demonstrates zkExp's scaling properties across deployment scenarios. Panel A confirms our theoretical $O(\ell \log \ell)$ complexity bounds, with hybrid FFT optimizations reducing practical overhead in the tested range (4 to 4096 bits). Verification time remains constant at 3.53–3.88ms across all exponent lengths, validating $O(1)$ cryptographic verification complexity. For cryptographically relevant 4096-bit exponents, zkExp achieves 16.3×

overhead compared to traditional square-and-multiply. Panel B illustrates the dramatic overhead reduction as exponent length increases, decreasing from 1,585× at 4 bits to 16.3× at 4096 bits. This behavior reflects the amortization of constant proof generation costs over larger computational work. Panel C reveals batch efficiency gains through our sliding-window aggregation protocol, achieving 1.35× overhead for 1000 exponentiations—an 87× improvement over single operations. Panel D shows memory crossover at approximately 60 operations, where zkExp's constant usage becomes more efficient than traditional linear scaling.

Table 4: **zkExp vs. Traditional Exponentiation Performance.** All zkExp configurations maintain 128-bit security. Throughput = total exponent bits processed per millisecond of proving time. Single proofs: 160B; batch proofs: 256B. **Bold** values highlight efficient deployment points: 16.3× overhead for cryptographic single exponents, 1.35× for large batches. Memory for traditional shown in MB (1MB = 10^6 bytes).

Type	Configuration	Time (ms)		Overhead	Throughput (kbps)	Proof (B)	Memory (MB)
		Prove	Verify				
Single	4-bit	1.59	3.53	1,585×	2.5	160	1.05
	256-bit	1.65	3.52	118×	155	160	1.06
	1024-bit	2.08	3.59	37×	493	160	1.08
	4096-bit	3.72	3.88	**16.3×**	**1,100**	160	1.11
Batch	10×256b	1.38	3.65	9.6×	1,862	256	1.06
	50×256b	2.55	4.20	3.6×	5,016	256	1.06
	100×256b	2.99	4.78	**2.1×**	8,562	256	1.06
	500×256b	10.11	9.69	1.4×	12,663	256	1.06
	1000×256b	19.30	15.77	**1.35×**	**13,263**	256	1.07
Sqr& Mult.	Single 256b	0.01	-	1.0×	25,600	N/A	0.00006
	Batch 1000×256b	14.32	-	1.0×	17,879	N/A	0.064
	Batch 100×1024b	6.09	-	1.0×	16,814	N/A	0.0064

Memory usage remains bounded at 1.06–1.11MB across all zkExp configurations, contrasting with traditional approaches that scale linearly from 0.00006MB to 0.064MB. Proof sizes are 160 bytes for single exponentiations (with pre-published commitments) and 256 bytes for batches, independent of exponent length or batch size. Throughput reaches 13.3 kbps for large batches, approaching theoretical limits of our sliding-window aggregation protocol. Verification time grows modestly from 3.5ms for single exponentiations to 15.8ms for large batches due to $O(T)$ field operations for window aggregation (where $T = \lceil 2k/w \rceil \approx 63$ for $k = 1000, w = 32$), while maintaining a single constant-cost pairing operation. This represents practical constant-time cryptographic verification compared to $O(k)$ cryptographic operations in traditional schemes.

These measurements identify practical deployment thresholds: single exponentiations become viable at $\ell \geq 1024$ bits (37× overhead), moderate batches achieve efficiency at $k \geq 50$ operations (3.6× overhead), and large-scale deployment approaches native performance at $k \geq 500$ operations (1.4× overhead).

8.6 Use Case: On-Chain Verification

Smart contract platforms impose economic constraints on cryptographic verification through gas-based cost models that penalize computationally intensive operations. We evaluate zkExp's efficiency for on-chain deployment, demonstrating how its constant-time cryptographic verification overcomes the linear-scaling costs of traditional schemes.

***Cost Model*:** Let k denote the number of exponentiation relations ($g^{x_i} = y_i$) being jointly verified, i.e., the batch size. Our cost analysis follows Ethereum's established

precompile pricing model under the London hardfork specification [BCD+21]. Ethereum provides precompiled contracts for cryptographic primitives with fixed gas costs: pairing-based verification via `bn254_pairing` costs 0.113M gas [Rei17], modular exponentiation via `ModExp` costs approximately 0.25M gas for 2048-bit operations [But17], and ECDSA verification via `ecrecover` requires 0.003M gas per signature [W+14]. These costs grow linearly in k, since each exponentiation or signature verification is charged individually.

In contrast, zkExp requires a single pairing operation, a hash over the public input, and transmission of a constant-size 256-byte proof. For a batch of k exponentiations, each contributing approximately 80 bytes to the hash input (one $\mathbb{G}_1$ base and one $\mathbb{F}_p$ result), the total gas cost is:

$$\text{Gas}_{\text{zkExp}} = \underbrace{0.113\text{M}}_{\text{pairing}} + \underbrace{60 \cdot \left\lceil \frac{80k}{32} \right\rceil}_{\text{hashing}} + \underbrace{16 \cdot 256}_{\text{calldata}} \approx 0.113\text{M} + 150 \cdot k + 4{,}096.$$

***Scaling Analysis*:** Table 5 compares gas costs across schemes. While ECDSA and RSA verification require k expensive cryptographic operations (each costing 0.003M–0.25M gas), zkExp requires only one pairing operation (0.113M gas) regardless of batch size, plus lightweight hashing and calldata costs that grow with k. This achieves $\mathcal{O}(1)$ cryptographic verification, since the pairing cost is constant, with overall gas growth coming only from lightweight hashing and calldata rather than repeated cryptographic operations. The crossover with ECDSA occurs at $k \approx 40$ exponentiations.

Table 5: **On-chain verification costs (M = million gas).** zkExp uses $\mathcal{O}(1)$ pairings plus lightweight $\mathcal{O}(k)$ hashing, while ECDSA/RSA require $\mathcal{O}(k)$ full cryptographic operations.

Protocol	**Gas Cost** ($k = 1$)	**Gas Cost** ($k = 1000$)	**Scaling**
zkExp	0.116M	0.267M	$\mathcal{O}(1)$ pairing
ECDSA Individual	0.003M	3M	$\mathcal{O}(k)$ operations
RSA-2048 Individual	0.25M	250M	$\mathcal{O}(k)$ operations

***Deployment and Applications*:** zkExp can be deployed using Ethereum's existing pairing precompiles. Future migration to BLS12-381 precompiles under EIP-2537 would enable enhanced security at reduced gas cost. Layer-2 solutions offer native BLS12-381 support with enhanced calldata efficiency. The constant-cost cryptographic verification enables efficient deployment of privacy-preserving protocols including anonymous credentials, threshold cryptographic systems, and large-scale verifiable computing. These applications become practical under zkExp's constant scaling where traditional linear-scaling approaches face prohibitive costs.

9 Conclusion

We presented zkExp, the first zero-knowledge proof system to achieve efficient asymptotic bounds for batched modular exponentiation: $\tilde{O}(k\ell)$ prover time, $O(1)$ verification time, and constant 256-byte proofs. By exploiting the algebraic structure of repeated squaring through polynomial commitments, zkExp overcomes fundamental limitations of classical batch verification and general-purpose SNARKs. Our protocol integrates four technical innovations: trace-based polynomial encoding, lazy sumcheck protocols, hybrid FFT decomposition, and sliding-window batching, that jointly enable constant-time verification for large-scale exponentiation statements. The implementation achieves 16.3× overhead for 4096-bit single exponentiations, reducing to 1.35× in 1000-batch settings, with 3.5 ms verification and under 80k gas regardless of batch size. zkExp enables practical deployment

in blockchain rollups, anonymous credentials, and threshold cryptography, where linear verification costs are prohibitive.

***Future Directions*:** Our current construction targets multiplicative groups, exploiting square-and-multiply structure. Natural extensions include generalizing to *scalar multiplication in additive groups* (elliptic curve operations), where double-and-add exhibits analogous recursive structure. This would enable efficient zero-knowledge proofs for ECDSA verification, threshold ECDH, and other elliptic curve primitives. Additional directions include optimizing polynomial commitment efficiency and extending to verifiable delay functions and distributed key generation. The design demonstrates that domain-specific algebraic structure can yield zero-knowledge proofs significantly more efficient than general-purpose constraint systems.

References

[AFK87] Martín Abadi, Joan Feigenbaum, and Joe Kilian. On hiding information from an oracle. In *Proceedings of the 19th Annual ACM Symposium on Theory of Computing (STOC)*, pages 195–203. ACM, 1987. doi:10.1145/28395.28417.

[BB04] Dan Boneh and Xavier Boyen. Short signatures without random oracles. In *Advances in Cryptology – EUROCRYPT 2004*, volume 3027 of *Lecture Notes in Computer Science*, pages 56–73. Springer, 2004. doi:10.1007/978-3-540-24676-3_4.

[BBB+18] Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell. Bulletproofs: Short proofs for confidential transactions and more. In *2018 IEEE Symposium on Security and Privacy*, pages 315–334. IEEE, 2018. doi:10.1109/SP.2018.00020.

[BBF19] Dan Boneh, Benedikt Bünz, and Ben Fisch. Batching techniques for accumulators with applications to iops and stateless blockchains. In *Advances in Cryptology – CRYPTO 2019*, volume 11692 of *Lecture Notes in Computer Science*, pages 561–586. Springer, 2019. doi:10.1007/978-3-030-26948-7_20.

[BCD+21] Vitalik Buterin, Eric Conner, Rick Dudley, Matthew Slipper, Ian Norden, and Abdelhamid Bakhta. Eip-1559: Fee market change for eth 1.0 chain. https://eips.ethereum.org/EIPS/eip-1559, 2021. Ethereum Improvement Proposal.

[BCG+13] Eli Ben-Sasson, Alessandro Chiesa, Daniel Genkin, Eran Tromer, and Madars Virza. Snarks for C: verifying program executions succinctly and in zero knowledge. In *Advances in Cryptology – CRYPTO 2013*, volume 8043 of *Lecture Notes in Computer Science*, pages 90–108. Springer, 2013. doi:10.1007/978-3-642-40084-1_6.

[BCR+19] Eli Ben-Sasson, Alessandro Chiesa, Michael Riabzev, Nicholas Spooner, Madars Virza, and Nicholas P. Ward. Aurora: Transparent succinct arguments for R1CS. In *Advances in Cryptology – EUROCRYPT 2019*, volume 11476 of *Lecture Notes in Computer Science*, pages 103–128. Springer, 2019. doi:10.1007/978-3-030-17653-2_4.

[BDN18] Dan Boneh, Manu Drijvers, and Gregory Neven. Compact multi-signatures for smaller blockchains. In *Advances in Cryptology – ASIACRYPT 2018*, volume 11273 of *Lecture Notes in Computer Science*, pages 435–464. Springer, 2018. doi:10.1007/978-3-030-03329-3_15.

[BFS20] Benedikt Bünz, Ben Fisch, and Alan Szepieniec. Transparent snarks from DARK compilers. In *Advances in Cryptology – EUROCRYPT 2020*, volume 12105 of *Lecture Notes in Computer Science*, pages 677–706. Springer, 2020. doi:10.1007/978-3-030-45721-1_24.

[BGH19] Sean Bowe, Jack Grigg, and Daira Hopwood. Halo: Recursive Proof Composition without a Trusted Setup. *IACR Cryptol. ePrint Arch.*, 2019:1021, 2019. URL: https://eprint.iacr.org/2019/1021.

[BGM17] Sean Bowe, Ariel Gabizon, and Ian Miers. Scalable Multi-party Computation for zk-SNARK Parameters in the Random Beacon Model. *IACR Cryptol. ePrint Arch.*, 2017:1050, 2017. URL: https://eprint.iacr.org/2017/1050.

[BHR+21] Alexander R. Block, Justin Holmgren, Alon Rosen, Ron D. Rothblum, and Pratik Soni. Time- and space-efficient arguments from groups of unknown order. In *Advances in Cryptology – CRYPTO 2021*, volume 12828 of *Lecture Notes in Computer Science*, pages 123–152. Springer, 2021. doi:10.1007/978-3-030-84259-8_5.

[BMM+21] Benedikt Bünz, Mary Maller, Pratyush Mishra, Nirvan Tyagi, and Psi Vesely. Proofs for inner pairing products and applications. In *Advances in Cryptology – ASIACRYPT 2021*, volume 13092 of *Lecture Notes in Computer Science*, pages 65–97. Springer, 2021. doi:10.1007/978-3-030-92078-4_3.

[BNN09] Mihir Bellare, Chanathip Namprempre, and Gregory Neven. Security proofs for identity-based identification and signature schemes. *J. Cryptol.*, 22(1):1–61, 2009. doi:10.1007/s00145-008-9028-8.

[BR93] Mihir Bellare and Phillip Rogaway. Random oracles are practical: A paradigm for designing efficient protocols. In *CCS '93: Proceedings of the 1st ACM Conference on Computer and Communications Security*, pages 62–73. ACM, 1993. doi:10.1145/168588.168596.

[BSBHR18] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Scalable, transparent, and post-quantum secure computational integrity. *IACR Cryptol. ePrint Arch.*, 2018:046, 2018. URL: https://eprint.iacr.org/2018/046.

[But17] Vitalik Buterin. Eip-198: Big integer modular exponentiation (modexp) precompile. https://eips.ethereum.org/EIPS/eip-198, 2017. Ethereum Improvement Proposal.

[Che10] Jung Hee Cheon. Discrete logarithm problems with auxiliary inputs. *J. Cryptol.*, 23(3):457–476, 2010. doi:10.1007/s00145-009-9047-0.

[CHM+20] Alessandro Chiesa, Yuncong Hu, Mary Maller, Pratyush Mishra, Noah Vesely, and Nicholas Ward. Marlin: Preprocessing zksnarks with universal and updatable srs. In *Advances in Cryptology – EUROCRYPT 2020*, volume 12105 of *Lecture Notes in Computer Science*, pages 738–768. Springer, 2020. doi:10.1007/978-3-030-45721-1_26.

[CHP07] Jan Camenisch, Susan Hohenberger, and Michael Østergaard Pedersen. Batch verification of short signatures. In *Advances in Cryptology – EUROCRYPT 2007*, volume 4515 of *Lecture Notes in Computer Science*, pages 246–263. Springer, 2007. doi:10.1007/978-3-540-72540-4_14.

[CL01] Jan Camenisch and Anna Lysyanskaya. An efficient system for non-transferable anonymous credentials with optional anonymity revocation. In *Advances in Cryptology – EUROCRYPT 2001*, volume 2045 of *Lecture Notes in Computer Science*, pages 93–118. Springer, 2001. doi:10.1007/3-540-44987-6_7.

[Com22] Electric Coin Company. Zcash protocol specification. https://zips.z.cash/protocol/protocol.pdf, 2022.

[CP19] Bram Cohen and Krzysztof Pietrzak. The chia network blockchain, 2019. White paper.

[DH25] Biniyam Deressa and M Anwar Hasan. zkmap: Zero-knowledge succinct non-interactive matrix multiplication proofs. *IACR Communications in Cryptology*, 2(3), 2025. doi:10.62056/angy11fgx.

[FKL18] Georg Fuchsbauer, Eike Kiltz, and Julian Loss. The algebraic group model and its applications. In *Advances in Cryptology – CRYPTO 2018*, volume 10991 of *Lecture Notes in Computer Science*, pages 33–62. Springer, 2018. doi:10.1007/978-3-319-96881-0_2.

[FKSX25] Xinxin Fan, Veronika Kuchta, Francesco Sica, and Lei Xu. Speeding up multi-scalar multiplications for pairing-based zksnarks. *J. Cryptol.*, 38(2):21, April 2025. doi:10.1007/s00145-025-09540-x.

[FS86] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In *Advances in Cryptology – CRYPTO 1986*, volume 263 of *Lecture Notes in Computer Science*, pages 186–194. Springer, 1986. doi:10.1007/3-540-47721-7_12.

[GMN22] Nicolas Gailly, Mary Maller, and Anca Nitulescu. Snarkpack: Practical SNARK aggregation. In *Financial Cryptography*, volume 13411 of *Lecture Notes in Computer Science*, pages 203–229. Springer, 2022. doi:10.1007/978-3-031-18283-9_10.

[Gol04] Oded Goldreich. *The Foundations of Cryptography - Volume 2: Basic Applications*. Cambridge University Press, 2004.

[Gro16] Jens Groth. On the size of pairing-based non-interactive arguments. In *Advances in Cryptology – EUROCRYPT 2016*, volume 9666 of *Lecture Notes in Computer Science*, pages 305–326. Springer, 2016. doi:10.1007/978-3-662-49896-5_11.

[GWC19] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. PLONK: permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge. *IACR Cryptol. ePrint Arch.*, 2019:953, 2019. URL: https://eprint.iacr.org/2019/953.

[HHI25] Charlotte Hoffmann, Pavel Hubácek, and Svetlana Ivanova. Practical batch proofs of exponentiation. *IACR Commun. Cryptol.*, 2(3):9, 2025. doi:10.62056/abvur-iuc.

[HKR19] Max Hoffmann, Michael Klooß, and Andy Rupp. Efficient zero-knowledge arguments in the discrete log setting, revisited. In *CCS*, pages 2093–2110. ACM, 2019. doi:10.1145/3319535.3354251.

[KST22] Abhiram Kothapalli, Srinath T. V. Setty, and Ioanna Tzialla. Nova: Recursive zero-knowledge arguments from folding schemes. In *Advances in Cryptology – CRYPTO 2022*, volume 13510 of *Lecture Notes in Computer Science*, pages 359–388. Springer, 2022. doi:10.1007/978-3-031-15985-5_13.

[KZG10] Aniket Kate, Gregory M Zaverucha, and Ian Goldberg. Constant-size commitments to polynomials and their applications. In *Advances in Cryptology – ASIACRYPT 2010*, volume 6477 of *Lecture Notes in Computer Science*, pages 177–194. Springer, 2010. doi:10.1007/978-3-642-17373-8_11.

[Lab20] Protocol Labs. Filecoin: A decentralized storage network. Technical report, Protocol Labs, 2020. URL: https://filecoin.io/filecoin.pdf.

[Nao03] Moni Naor. On cryptographic assumptions and challenges. In *CRYPTO*, volume 2729 of *Lecture Notes in Computer Science*, pages 96–109. Springer, 2003. doi:10.1007/978-3-540-45146-4_6.

[Rei17] Christian Reitwießner. Eip-197: Precompiled contracts for optimal ate pairing check on the elliptic curve alt_bn128. https://eips.ethereum.org/EIPS/eip-197, 2017. Ethereum Improvement Proposal.

[Rot21] Lior Rotem. Simple and efficient batch verification techniques for verifiable delay functions. In *Theory of Cryptography Conference (TCC) 2021*, volume 13044 of *Lecture Notes in Computer Science*, pages 382–414. Springer, 2021. doi:10.1007/978-3-030-90456-2_13.

[RSA78] Ronald L. Rivest, Adi Shamir, and Leonard M. Adleman. A method for obtaining digital signatures and public-key cryptosystems. *Commun. ACM*, 21(2):120–126, February 1978. doi:10.1145/359340.359342.

[Sch80] Jacob T Schwartz. Fast probabilistic algorithms for verification of polynomial identities. *Journal of the ACM (JACM)*, 27(4):701–717, 1980. doi:10.1145/322217.322225.

[Sch91] Claus-Peter Schnorr. Efficient signature generation by smart cards. *J. Cryptol.*, 4(3):161–174, January 1991. doi:10.1007/BF00196725.

[Set20] Srinath T. V. Setty. Spartan: Efficient and general-purpose zksnarks without trusted setup. In *Advances in Cryptology – CRYPTO 2020*, volume 12172 of *Lecture Notes in Computer Science*, pages 704–737. Springer, 2020. doi:10.1007/978-3-030-56877-1_25.

[Sho97] Victor Shoup. Lower bounds for discrete logarithms and related problems. In *Advances in Cryptology – EUROCRYPT 1997*, volume 1233 of *Lecture Notes in Computer Science*, pages 256–266. Springer, 1997. doi:10.1007/3-540-69053-0_18.

[W+14] Gavin Wood et al. Ethereum: A secure decentralised generalised transaction ledger. *Ethereum project yellow paper*, 151(2014):1–32, 2014.

[Wes19] Benjamin Wesolowski. Efficient verifiable delay functions. In *Advances in Cryptology – EUROCRYPT 2019*, volume 11478 of *Lecture Notes in Computer Science*, pages 379–407. Springer, 2019. doi:10.1007/978-3-030-17659-4_13.

[WTS+18] Riad S Wahby, Ioanna Tzialla, Abhi Shelat, Justin Thaler, and Michael Walfish. Doubly-efficient zksnarks without trusted setup. In *2018 IEEE Symposium on Security and Privacy*, pages 926–943. IEEE, 2018. doi:10.1109/SP.2018.00060.

[ZCC+16] Fan Zhang, Ethan Cecchetti, Kyle Croman, Ari Juels, and Elaine Shi. Town crier: An authenticated data feed for smart contracts. In *CCS*, pages 270–282. ACM, 2016. doi:10.1145/2976749.2978326.